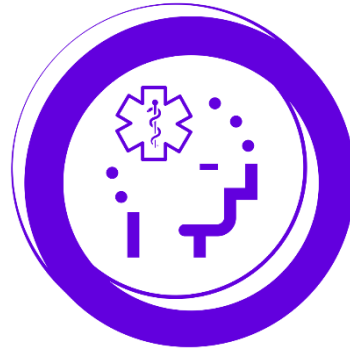




**SUEZ UNIVERSITY
FACULTY OF COMPUTERS
AND
INFORMATION**



Seravian

AI-Powered Mental Health Assistant Platform

Abdalrhman Mohamed Mohamed	210900012
Ahmed Gharieb Mansour	210900015
Hossam Walid Elsayed Ahmed	210900045
Kareem Essam Saadallah	210900047
Khaled Mohamed Said	210900064
Mohamed Saeed Goda	210900013

Under the Supervision of

DR. FAYZA NADA

&

DR. RANIA SALAMA

**Department of Computer Science
2024 - 2025**

الرؤية والرسالة

رؤية الكلية

أن تكون الكلية منارة في تقديم البرامج الدراسية والأبحاث والاستشارات في مجالات علوم الحاسب والمعلومات والذكاء الاصطناعي على المستوى المحلي والإقليمي.

رسالة الكلية

تسعى كلية الحاسبات و المعلومات - جامعة السويس لتكون مركزاً لتطوير برامج الدراسة والبحث وتقديم الاستشارات في علوم الحاسب وتكنولوجيا ونظم المعلومات والذكاء الاصطناعي وإعداد الكوادر ذات الكفاءة العالية القادرة على المنافسة.



DECLARATION

We hereby declare that this dissertation has been carried out under our supervision. To the best of our knowledge, the work is original, and the students have undertaken it with integrity and academic honesty.

Supervisors

Signature

Dr. Fayza Nada

Dr. Rania Salama

We hereby declare that our dissertation is entirely our work and genuine / original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade and it may result in withdrawal of our Bachelor's degree.

Name

Signature

Abdalrhman Mohamed Mohamed

Ahmed Gharieb Mansour

Hossam Walid Elsayedahmed

Kareem Essam Saadallah

Khaled Mohamed Said

Mohamed Saeed Goda



ACKNOWLEDGMENT

First and foremost, we thank and praise Allah for His countless blessings. He granted us the strength, determination, patience, and time needed to complete this work. Without His guidance, we would not have reached where we are today, and we would have found ourselves in a situation we could not have managed.

We also wish to express our heartfelt gratitude and deep appreciation to all those—individuals and institutions—who supported us throughout our academic journey and played a significant role in facilitating and enriching this project.

We extend our sincere thanks and profound appreciation to our esteemed supervisors, Dr. Fayza Nady and Dr. Rania Salama, for their continuous support, valuable guidance, and constructive feedback. Their advice stemmed from profound experience and insight, and their encouragement and dedication to the quality of our work had a remarkable impact on the development of this project. We could not have hoped for better supervision.

We must also express our deepest thanks to our families and friends, who have always been our source of strength. Through their prayers, unconditional support, and genuine love, we were able to overcome many challenges. Words of thanks cannot truly express our appreciation, but we sincerely pray that Allah rewards them generously and grants them lasting health and happiness.



ABSTRACT

Mental health has become an increasingly critical aspect of overall well-being, yet many individuals face barriers when seeking support, including social stigma, limited access to professionals, and time constraints. This project presents a comprehensive AI-powered mental health platform consisting of an Android mobile application and a web interface, designed to provide accessible, empathetic, and intelligent mental health support through both automated and professional interactions.

The platform enables users to register and identify as either patients or doctors. Patients can interact with a smart AI chatbot for emotional support and early diagnosis, and they can also book online consultations with qualified mental health professionals through a seamless virtual interface. The system is built to ensure privacy, inclusivity, and emotional sensitivity at every stage of interaction.

At the core of the system is a set of advanced artificial intelligence models and tools. This integration of multimodal AI technologies allows the platform to interpret text, voice, and facial cues, simulating a human-like interaction tailored to the user's mental state. The system aims to act as a supportive companion while offering a bridge to real professional help when needed.

In conclusion, the project demonstrates how modern AI can be effectively harnessed to support mental health at scale. It provides a practical, scalable, and emotionally intelligent solution that contributes to improving access to mental health care, especially in underserved or remote communities.



Table of Contents

Acknowledgement	3
Abstract	4
Chapter One: Introduction & Background	7
1.1 Introduction	9
1.2 Background	9
1.3 Problem Statement	10
1.4 Objectives of the Project	10
1.5 Mental Health Challenges and the Role of Technology	11
1.6 Importance of AI in Mental Health Support	11
1.7 AI Technologies Used	12
1.7.1 Transformers (AutoTokenizer, AutoModelForCausalLM)	12
1.7.2 Whisper (Voice-to-Text)	12
1.7.3 DeepFace (Emotion Recognition)	13
1.7.4 MentalLama Chat 7B	13
1.7.5 BitsAndBytes (Quantization for Model Optimization)	14
1.7.6 Whisper (Speech Emotion Recognition)	14
1.7.7 Tortoise-TTS (Text-to-Speech)	15
Chapter Two: Proposal Work, Requirement Specification & System Analysis	16
2.1 Proposed System Overview	17
2.2 Functional Requirements	18
2.3 Non-Functional Requirements	18
2.4 System Users and Roles	19
2.5 Use Case Diagram	20
2.6 System Block Diagram	26
2.7 ERD (Entity Relationship Diagram)	30
2.8 Tools and Technologies Used	36
2.8.1 Mobile Application	36
2.8.2 Web Application	37
2.8.3 Back-end	38
2.8.4 AI	40



Chapter Three: Implementation and results	41
3.1 Front-End Implementation	42
3.1.1 Android Application	42
3.1.1.1 Sign up	42
3.1.1.2. OTP	43
3.1.1.3 Login	44
3.1.1.4 Profile setup	45
3.1.1.5 Main App Structure	46
3.1.2 Web Application	49
3.1.2.1 Sign up	49
3.1.2.2. OTP	50
3.1.2.3 Profile Setup	51
3.1.2.4 Login	57
3.1.2.5 Main Structure	59
3.2 Back-End Implementation	64
3.2.1 Database	64
3.2.2 RESTful API Development (ASP.NET Core)	64
3.2.3 SignalR-Based Real-Time Chat	64
3.2.4 Emails & OTP Verification	65
3.2.5 JWT Authentication System	65
3.2.6 AI Models Integrations	66
3.2.7 Hosting and Deployment	67
3.3 AI Models Integration	68
3.3.1 Chatbot with LLM	68
3.3.2 Speech Recognition (Whisper)	75
3.3.3 Emotion Analysis (DeepFace)	78
3.3.4 Speech Emotion Recognition (Whisper Fine tuned)	79
3.3.5 Tortoise-TTS (Text-to-Speech)	82



Chapter Four: Conclusion and Future Work	86
4.1 Conclusion	87
4.2 Challenges Faced	87
4.3 Limitations of the System	88
4.4 Future Enhancements	88
References	90
Appendix	91
Arabic Summary	98



CHAPTER ONE:

Introduction & Background



1.1. Introduction

Mental health disorders have emerged as a major public health concern, affecting millions of individuals across the globe across all age groups. In developing and lower-middle-income countries, the situation is particularly severe due to widespread stigma, lack of awareness, and a chronic shortage of mental health infrastructure. Despite growing recognition of mental health as a vital component of overall well-being, access to quality care remains deeply inadequate and often out of reach for those who need it most.

According to the World Health Organization, the treatment gap for mental disorders in developing countries ranges between 76% and 85%. In India, the National Mental Health Survey reports a treatment gap as high as 83%, highlighting the vast number of individuals who suffer in silence without proper diagnosis, support, or treatment. This is further compounded by a critical shortage of trained professionals, inequitable distribution of services, and significant financial, social, and geographical barriers that prevent people—especially in rural, remote, or underserved communities—from seeking timely help.

The consequences of untreated mental health conditions are wide-ranging and deeply impactful. Individuals often experience worsening symptoms, social isolation, reduced quality of life, and higher risks of co-occurring disorders. At the societal level, the burden includes increased healthcare costs, diminished workforce productivity, and long-term economic and social impact. These persistent challenges underscore an urgent need for scalable, inclusive, and culturally sensitive mental health care solutions that can bridge the existing gaps and deliver timely, effective support.

1.2. Background

To address the growing mental health crisis, especially in resource-limited settings, technology-driven solutions have emerged as a promising way to increase accessibility and affordability of mental health care. Digital platforms, such as mobile apps, web-based programs, and conversational agents, offer scalable solutions that can bridge gaps in care, reduce stigma, and overcome barriers such as geographic location and limited professional availability.

Text-based chatbots, for example, provide a unique opportunity to engage individuals in meaningful conversations about mental health, allowing for early screening, self-assessment, and even ongoing support. These tools are designed to mimic human interaction, offering comfort and companionship to users who may otherwise face challenges accessing traditional mental health services.

As the demand for mental health services continues to grow, the use of such technologies could play a critical role in expanding care delivery. The potential of AI-based chatbots to deliver timely and accessible mental health assessments makes them a valuable asset in creating more inclusive, effective, and scalable mental health care systems worldwide.



1.3 Problem Statement

In many developing and lower-middle-income countries, mental health care remains largely inaccessible due to a severe shortage of professionals, limited infrastructure, and widespread stigma. This has led to a significant treatment gap, with the majority of individuals unable to receive timely assessment or support. There is an urgent need to identify effective, scalable solutions that can accurately assess mental health conditions and reach underserved populations without relying on traditional, resource-heavy methods.

1.4 Objectives of the Project

Provide AI-powered mental health support through an intelligent chatbot:

The platform integrates a large language model (LLM) specifically trained on mental health-related conversations to provide users with a supportive, emotionally aware chatbot. This chatbot simulates human-like dialogue, listens to users' problems, offers calming responses, and provides early guidance or coping strategies. By enabling 24/7 availability, the chatbot serves as a virtual companion for users seeking immediate emotional support, especially during times when professional help is not readily accessible.

Enable online booking with certified mental health professionals for virtual consultations:

To bridge the gap between users and professional care, the system includes a doctor-patient module where users can browse available mental health professionals and schedule **online** sessions via video call. This reduces the barrier of geographical constraints and physical appointments, allowing users to get professional help from the comfort of their homes. The system handles appointment management, availability tracking, and notification alerts to streamline the process.

Optimize AI models using BitsAndBytes to ensure efficient performance on mobile and web platforms:

Large language models are often resource-intensive and difficult to deploy on standard devices. This project addresses that challenge using **BitsAndBytes**, a library that enables model quantization. Quantization reduces the size and computational cost of LLMs without sacrificing much performance. This optimization allows the chatbot and other AI features to run smoothly on mobile phones and web browsers, ensuring low-latency responses and minimal resource consumption.

Ensure user privacy and data security during conversations and consultations:

Given the sensitive nature of mental health, the system is designed with strict privacy and security standards. User data, including chat logs and medical information, is stored securely and encrypted. JWT (JSON Web Token) authentication is used to protect user sessions, and access control mechanisms ensure that only authorized users (e.g., doctors) can view sensitive information. All communication channels, especially during video consultations, are secured to maintain confidentiality.



Design a user-friendly interface accessible on both Android and web platforms for maximum usability:

The platform features a clean, intuitive interface tailored for ease of use across all age groups. Whether accessed via Android app or web browser, the UI is responsive and accessible, following modern design practices. Key functionalities such as chatbot access, booking appointments, and viewing doctor profiles are easily navigable. Special attention is given to user experience (UX), making mental health support more approachable and less intimidating for first-time users.

1.5 Mental Health Challenges and the Role of Technology

The integration of digital technology into mental health care is transforming how individuals access and experience psychological support. Teletherapy, along with interactive mobile applications and online counseling platforms, is enabling remote and real-time mental health services. These tools offer users privacy, accessibility, and flexibility—allowing them to receive guidance and engage in therapeutic practices without the need for in-person visits. Especially in underserved areas, teletherapy bridges critical gaps by connecting individuals to professionals and resources that would otherwise be unavailable.

In addition, technology-driven mental health solutions are capable of collecting and analyzing user data to support early detection and personalized care. They provide scalable ways to deliver psychoeducation, self-help exercises, and emotional support in user-friendly formats. This digital evolution is reshaping mental health systems into more inclusive, responsive, and user-centered models. As teletherapy and related technologies continue to advance, they hold the promise of enhancing the reach, efficiency, and effectiveness of mental health care worldwide.

1.6 Importance of AI in Mental Health Support

Artificial Intelligence (AI) is revolutionizing mental health care by enabling early detection of mental health disorders, personalizing treatment plans, and facilitating continuous monitoring of patients' mental states. AI algorithms can analyze vast amounts of data, including behavioral patterns and physiological signals, to identify subtle signs of mental health issues, allowing for timely interventions. This proactive approach not only improves patient outcomes but also reduces the burden on mental health professionals by streamlining diagnostic and treatment processes .

Furthermore, AI enhances teletherapy by providing tools that support remote mental health services, making care more accessible to individuals in underserved or remote areas. AI-driven platforms can offer real-time feedback, adapt therapeutic approaches based on patient responses, and ensure consistent monitoring, thereby improving the efficacy of teletherapy sessions. By integrating AI into teletherapy, mental health services become more scalable and responsive to individual needs, addressing the growing demand for mental health support .



1.7 AI Technologies Used

1.7.1 Transformers

(AutoTokenizer, AutoModelForCausalLM)

Transformers are deep learning models that utilize self-attention mechanisms and are widely used in NLP tasks. The Hugging Face Transformers library provides access to a vast collection of pre-trained models and components, enabling developers to apply state-of-the-art models to a variety of NLP tasks.

How It Works:

AutoTokenizer converts raw text into tokenized data, breaking words into smaller subwords or tokens.

Handles padding, truncation, and special tokens like [CLS] and [SEP].

AutoModelForCausalLM loads models designed for causal language modeling (e.g., GPT-2, GPT-3) to predict the next word in a sequence, ideal for generating text-based responses.

Example Model Used:

GPT-2, LLaMA, Falcon, or your fine-tuned MentalLam 7B model.

Advantages of Transformers

1. **Contextual understanding:** Enables the generation of coherent, context-aware responses for chatbots.
2. **Wide accessibility:** Pre-trained models are available for numerous applications, reducing the need to train models from scratch.
3. **Scalability:** Works with various transformer architectures like GPT-2, LLaMA, and Falcon.

1.7.2 Whisper (Voice-to-Text)

Whisper is an automatic speech recognition (ASR) model by OpenAI, capable of transcribing audio in over 90 languages. It uses a Transformer-based encoder-decoder architecture and is trained on a massive amount of multilingual data.

How It Works:

The encoder transforms audio spectrograms into embeddings.

The decoder generates the text transcript, with additional features like language detection and translation.



Advantages of Whisper

1. **Multilingual support:** Handles over 90 languages, making it useful globally.
2. **Robustness:** Effective in noisy environments and with multiple speakers.
3. **Real-time transcription:** Can transcribe live audio with high accuracy.

1.7.3 DeepFace (Emotion Recognition)

DeepFace is a facial recognition and emotion analysis framework developed by Facebook AI. It detects and analyzes facial expressions to infer emotions, helping apps understand and respond to human feelings.

How It Works:

Uses CNN-based models like VGG-Face and Dlib for face detection.

Analyzes facial muscle movements and expressions to recognize emotions such as happy, sad, angry, or neutral.

Advantages of DeepFace

1. **Real-time analysis:** Provides instant emotion detection, making it suitable for interactive applications.
2. **Emotion-aware responses:** Enhances user experience by adapting the system's tone based on detected emotions.
3. **Integration-friendly:** Easily integrates into mobile apps with lightweight models.

1.7.4 MentalLama Chat 7B

MentalLama Chat 7B is a large language model (LLM) specifically fine-tuned for mental health applications. It is trained to provide empathetic, safe, and informative responses, especially related to sensitive mental health topics like anxiety, depression, and stress.

How It Works:

Pre-trained on general language data and further fine-tuned on mental health-specific data, including therapeutic language and crisis handling scenarios.

Designed to simulate the response style of a therapist, providing guidance and emotional support.



Advantages of MentalLama

1. **Sensitive and safe interactions:** Specifically designed to avoid harmful or false medical advice.
2. **Empathy-driven responses:** Tailored for providing emotional validation and support.
3. **Best practices:** Aligns responses with psychological best practices for therapy.

1.7.5 BitsAndBytes (Quantization for Model Optimization)

BitsAndBytes is a library that facilitates 8-bit and 4-bit quantization of large models, helping compress models to reduce memory requirements. It is widely used to make transformer models more efficient and deployable on consumer hardware.

How It Works:

Converts 32-bit floating-point model weights into 8-bit or 4-bit integers.

Reduces GPU memory usage, allowing large models to run on less powerful hardware.

Advantages of BitsAndBytes

1. **Efficient memory usage:** Reduces memory consumption by up to 75%, enabling large models to run on consumer-grade hardware.
2. **Minimal performance loss:** The quantization results in only a slight decrease in model performance (~1–2%).
3. **Scalability:** Makes it possible to deploy large models on mobile devices and cloud VMs without needing expensive infrastructure.

1.7.6 Speech Emotion Recognition with Whisper Large v3

Speech Emotion Recognition with Whisper Large-v3 is a fine-tuned deep learning model that detects emotions directly from audio recordings.

It builds upon OpenAI's Whisper Large-v3 — one of the most powerful speech models — and specializes it for emotion classification without requiring manual transcription

How it works:

This system performs speech emotion recognition directly from raw audio files (e.g., .wav, .mp3, .m4a). It uses Whisper Large-v3 to extract deep audio features like tone and pitch, then classifies emotions without needing transcripts. The output is a ranked list of emotions with confidence scores.



Advantages:

1. No transcription required
2. Built on Whisper Large-v3 for robust audio feature extraction
3. Multilingual and accent-tolerant
4. Accepts various audio formats
5. Easy integration via Hugging Face's audio-classification pipeline
6. Ideal for emotion analysis, live feedback systems, or tracking emotional trends.

1.7.7. Tortoise-TTS(Text-to-Speech)

Tortoise TTS Integration

Tortoise TTS is a high-quality, multi-voice text-to-speech system developed by [neonbjb](#). It is integrated into this project to provide **ultra-realistic, expressive speech synthesis**.

Overview

Tortoise focuses on **maximum voice fidelity** using a multi-stage pipeline involving:

- An **autoregressive transformer** for base speech generation.
- A **diffusion model** for refining audio features.
- A **HiFi-GAN vocoder** for waveform synthesis.

It supports voice cloning, multi-voice blending, and custom speaker conditioning.

Features

- Realistic voices with emotional nuance.
- Multi-speaker support and conditioning.
- Open weights, no API key required.
- Voice cloning with a few sample clips.

Limitations

- **Slow generation** time due to quality-first architecture.
- Requires **GPU acceleration** for practical use.
- Not designed for real-time or mobile use cases.



CHAPTER TWO:

Proposal Work, Requirement Specification & System Analysis



2.1 Proposed System Overview

The proposed system is an AI-powered mental health platform designed to support users emotionally and psychologically through a combination of intelligent chatbot interactions, emotional state analysis, and real-time online consultations with licensed psychiatrists. The platform is accessible via both Android mobile application and web interface, offering 24/7 availability and a personalized experience across devices.

Core Features of the Proposed System:

1. AI Chatbot with Emotional Intelligence

The system uses a fine-tuned large language model (LLM), MentalLam Chat 7B, to generate emotionally sensitive, empathetic responses in natural conversations. It processes not only textual inputs but also integrates voice tone (via Whisper) and facial expression analysis (via DeepFace) to evaluate the user's emotional state in real-time. This makes responses more personalized, therapeutic, and human-like.

2. Real-Time Online Consultations

Users can easily schedule secure video sessions with licensed mental health professionals. The system includes therapist discovery features, allowing users to select doctors based on ratings, pricing, specialization, and availability. These consultations happen within the platform, with integrated secure video and payment systems.

3. Emotion Analysis via Multimodal Inputs

The system analyzes emotional signals from:

- Text inputs using NLP models.
- Voice using the Whisper ASR model to understand speech tone.
- Face using DeepFace to detect visible expressions. This multimodal analysis improves diagnosis suggestions and enhances the relevance of the chatbot's responses.

4. Multi-Platform Accessibility

The platform is built to be light and efficient, thanks to model quantization via BitsAndBytes, allowing it to run on both low-end and high-end devices. It is available on:

- Mobile (Kotlin-based Android App)
- Web (Angular Web App)

5. Secure and Scalable Backend

The backend, developed using ASP.NET, handles:

- Secure session management
- Video call routing
- Chat and emotion data processing
- Authentication via JWT
- Payments, using encrypted channels



6. Cloud Deployment and Continuous Development

- Hosted on scalable cloud platforms such as AWS or Azure.
- Codebase is managed through GitHub with Agile practices using Huly for planning and iteration.
- Testing includes unit, integration, and user testing for quality assurance.

Innovation and Impact

- **Personalized Emotional Insights:** Unlike traditional apps, this system captures emotional context from tone and facial expressions, not just text.
- **Always Available:** The AI chatbot operates 24/7, bridging gaps between therapy sessions or when users are in urgent need.
- **Affordable and Inclusive:** The AI assistant provides immediate support at no cost, while online professional sessions are available at a reduced cost compared to traditional therapy.

2.2 Functional Requirements

These define what the system should do the core features and functionalities.

1. User Authentication

- Users (patients and doctors) must be able to sign up and log in securely.
- Use of JWT tokens for session management.

2. Chatbot Interaction

- Users can chat with the AI-powered mental health assistant.
- The chatbot should generate context-aware, empathetic responses using the LLM.
- Supports both text and voice input via the Whisper model.

3. Emotion Detection

- Detect emotional state via:
 - Facial expression analysis (DeepFace)
 - Vocal tone analysis (voice input)
- Adapt chatbot responses accordingly.

4. Admin Panel (Optional)

- Admins can manage doctor accounts, user complaints, and monitor system performance.

5. Multi-Platform Access

- Functional on both mobile (Android) and web platforms.

2.3 Non-Functional Requirements

These define how the system performs—quality attributes like performance, usability, and security.

1. Usability

- Clean, intuitive UI for both patients and doctors.
- Easy navigation for all ages and backgrounds.



2. Scalability

- System must scale to support multiple users simultaneously (especially the chatbot).

3. Performance

- Response time for chatbot must be under 2 seconds.

4. Security

- User data must be encrypted (especially chat, emotion logs, and consultation history).
- Secure JWT-based authentication.

5. Availability

- Chatbot should be available 24/7 without downtime.

6. Maintainability

- Code should follow modular, well-documented architecture to ease updates and bug fixing.

7. Cross-Platform Compatibility

- The web version should be responsive (desktop, tablet, mobile browsers).
- The Android app should support Android 8.0+.

8. Localization (Optional)

- Support for multiple languages (e.g., Arabic and English) to expand reach.

2.4 System Users and Roles

The system supports multiple types of users, each with specific roles and access permissions based on their interaction with the platform. The users are classified into three main categories:

1. Patient (End User)

- Can register and log in to the platform.
- Interacts with the AI chatbot for emotional support.
- Inputs voice or text messages, and optionally enables camera for emotion detection.
- Views therapist profiles (ratings, availability, pricing).
- Makes secure online payments for booked sessions.
- Views previous session history and chatbot logs.

2. Doctor (Mental Health Professional)

- Registers and logs in as a verified psychiatrist.
- Updates profile, specialization, availability, pricing, and consultation schedule.
- Views AI-generated emotion insights (from facial/vocal analysis) to better understand patient state.

3. Admin (Optional – System Manager)

- Manages doctor approvals and account verifications.
- Monitors system usage and performance.



- Handles user complaints and feedback.
- Manages reports, logs, and security alerts.
- Oversees payment disputes or technical issues (if any).

2.5 Use Case diagram

The use case diagram provides a high-level representation of the interactions between users (actors) and the system. It highlights the main functionalities that the system offers and the roles that different users (such as patients, doctors, and admins) play within the platform.

In this project, the use case diagram illustrates how users can interact with key system features such as registering, logging in, chatting with the AI assistant, booking online consultations, making secure payments, and managing appointments. It also shows how doctors handle sessions and how the system ensures secure and seamless communication between all parties.

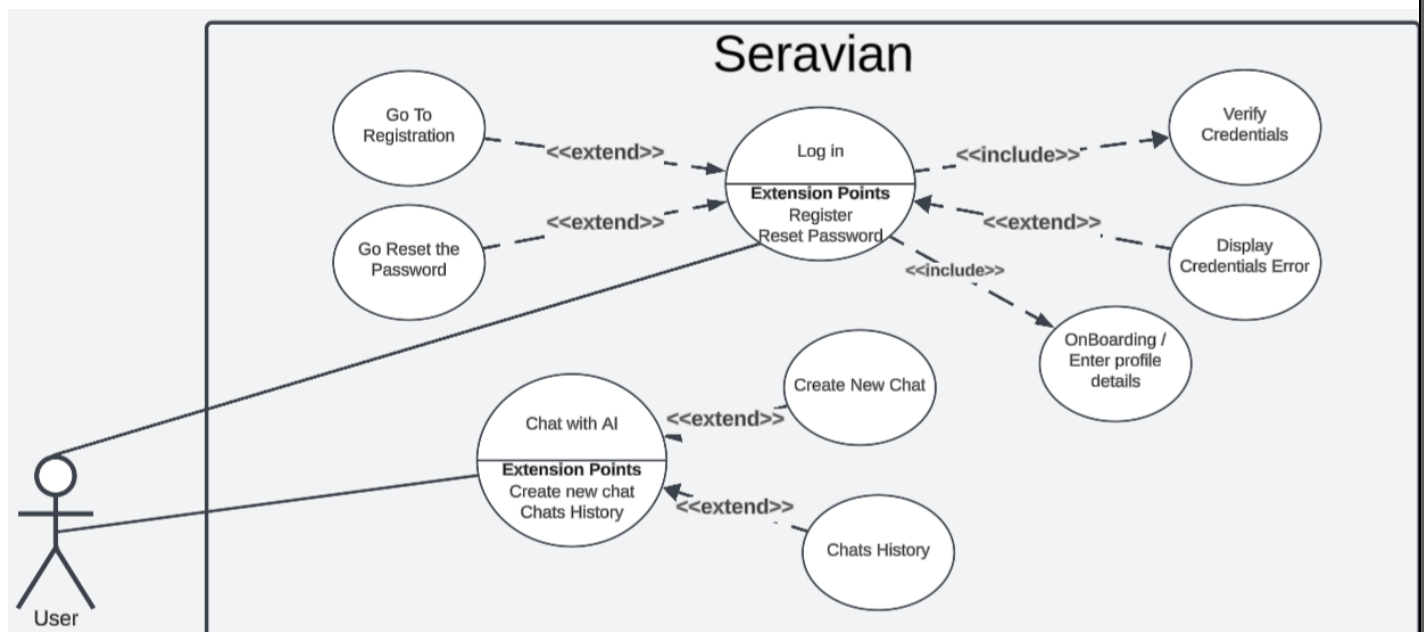
Actors

Patient: -

A person seeking mental therapy sessions, or AI-based conversation.

Doctor: -

A person helping patients to recover from their mental illness.



Seravian User Authentication System Description: Provides user authentication, account management, and AI chat functionality within the Seravian application.

Preconditions:



- System is accessible and operational
- Database connection is established
- Authentication services are functional

Main Flow:

1. User accesses the Seravian application
2. User can choose between:
 - Login with existing credentials
 - Register for a new account
 - Reset forgotten password
3. For login:
 - System verifies credentials
 - If valid, user gains access to application features
 - If invalid, system displays appropriate error message
4. After successful authentication:
 - User can create new chat sessions
 - User can view chat history
 - User can manage profile details
5. User interacts with AI through the chat interface

Postconditions:

- User is authenticated in the system
- User has access to application features based on privileges
- User can interact with the AI chat functionality
- Session data is stored securely

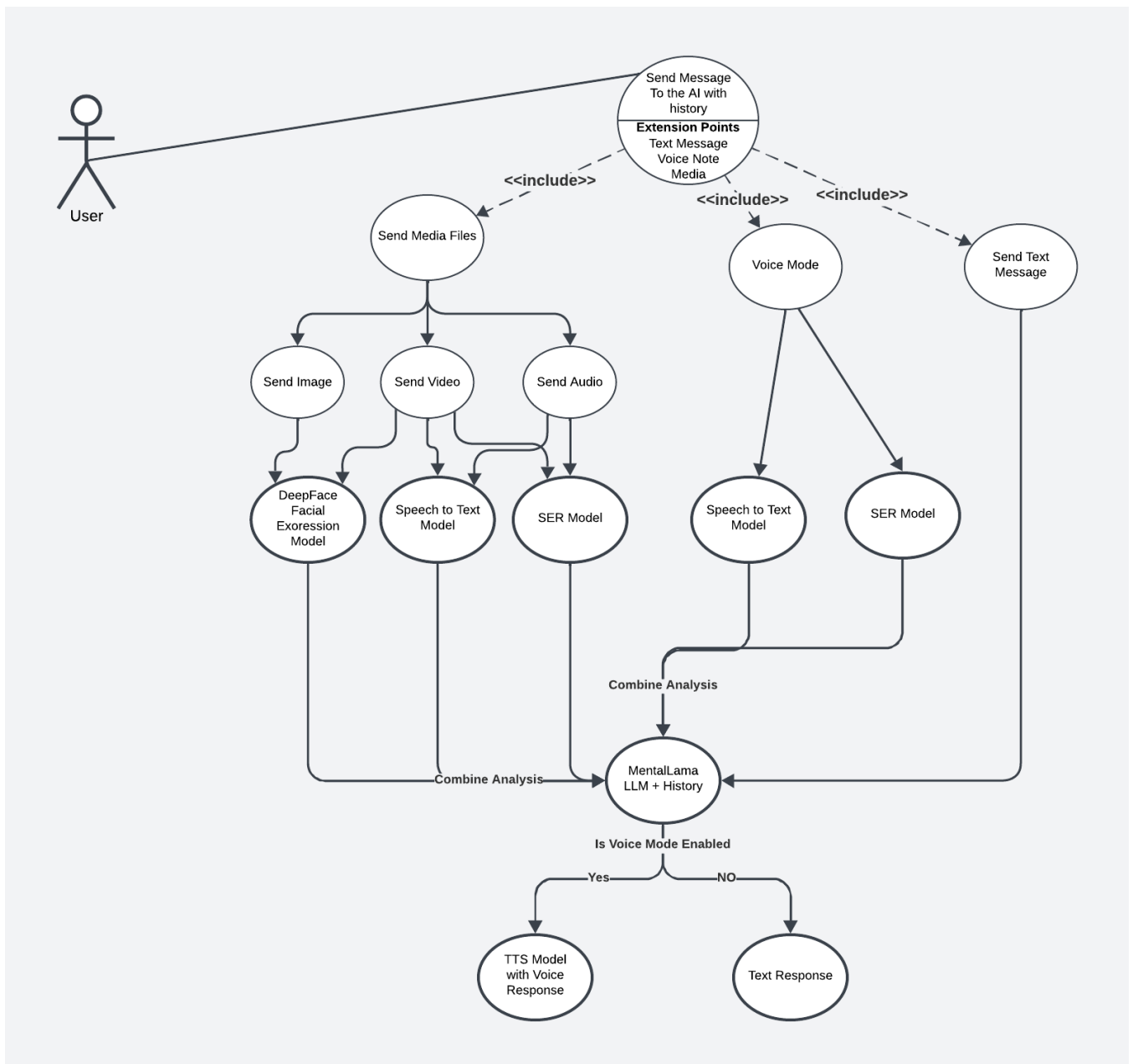
Alternative Flows:

- If credentials are invalid, system shows error message and prompts for retry
- If user forgets password, reset password flow is initiated
- If profile details are incomplete, user is prompted to complete them

Notes:

- Authentication uses secure token-based system
- Chat history is maintained for user convenience
- Profile details are required for personalized AI interaction
- System provides appropriate error handling for all authentication failures





AI Communication System

Description: Enables users to interact with an AI system through various media formats including text, voice, and multimedia files, with the system processing inputs through specialized models and generating appropriate responses.

Actors Involved: User, AI System

Preconditions:

- User has access to the AI communication platform
- AI system is operational with all required models available
- Network connectivity is established



Main Flow:

1. User initiates communication with the AI system
2. User can choose between sending:
 - Text messages
 - Media files (image, video, audio)
 - Voice notes
3. The system processes inputs through appropriate models:
 - Text is analyzed directly
 - Images are processed via facial expression models
 - Video uses speech-to-text and S2R models
 - Audio uses speech-to-text and S2R models
 - Voice notes use speech-to-text and S2R models
4. All inputs are combined for comprehensive analysis through InternalLLMs and history-aware processing
5. System generates responses based on analysis and user preferences

Postconditions:

- User receives appropriate response (text or voice-based)
- Communication history is updated
- Analysis results are stored for context in future interactions

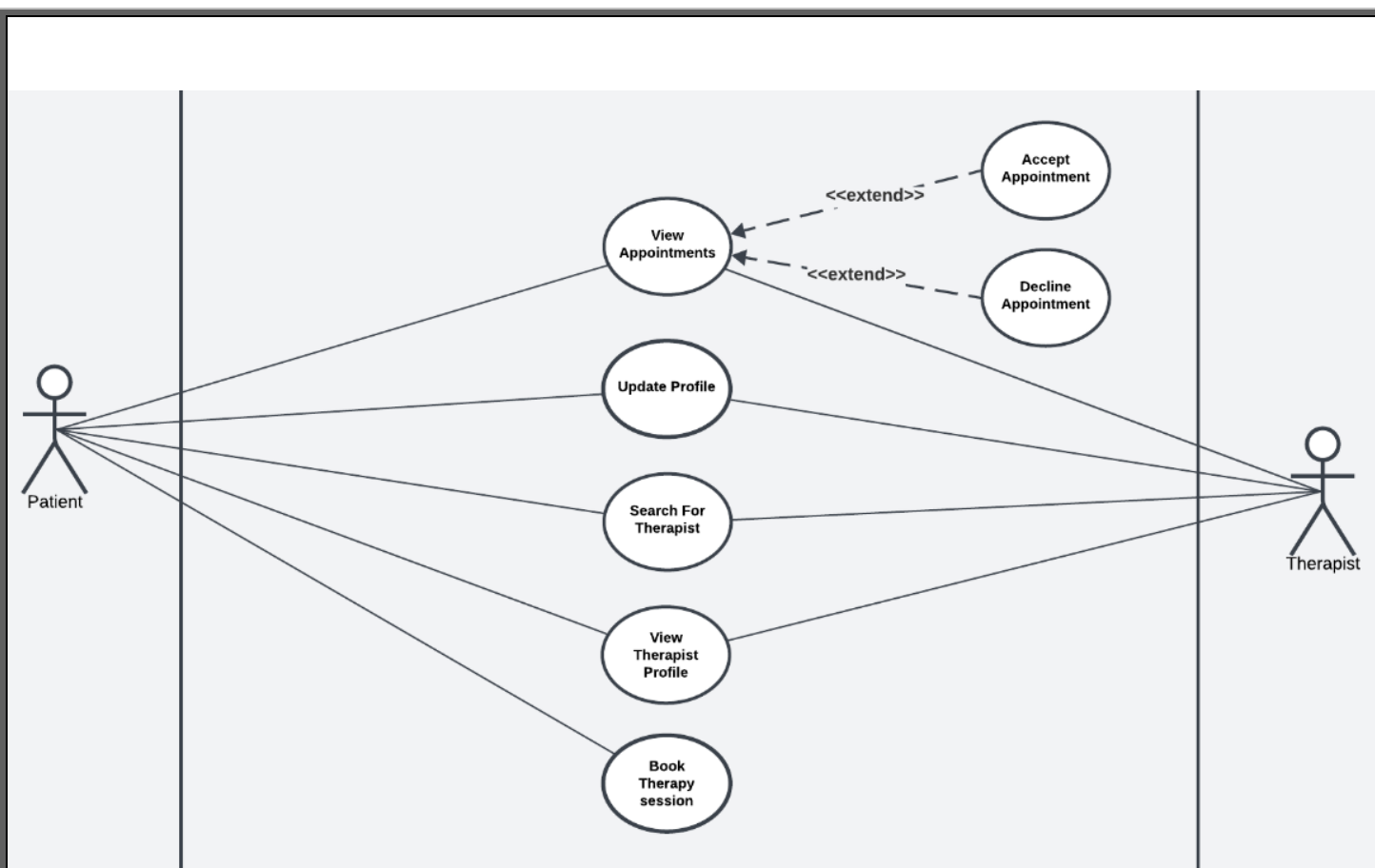
Alternative Flows:

- If voice mode is enabled, responses use TTS (Text-to-Speech) model
- If voice mode is disabled, text responses are provided

Notes:

- System maintains conversation history for contextual understanding
- Multiple processing models work in parallel to analyze different input types
- Final response generation considers all available context and user preferences





Therapy Session Management System Description: Enables patients to search for therapists, book appointments, and manage therapy sessions through an integrated platform.

Actors Involved: Patient, Therapist

Preconditions:

- Patient has registered and logged into the system
- Therapist profiles are available in the system
- Scheduling system is operational

Main Flow:

1. Patient accesses the therapy session management system
2. Patient can choose between:
 - Viewing existing appointments
 - Updating personal profile
 - Searching for therapists
 - Viewing therapist profiles
 - Booking therapy sessions
3. For appointment management:
 - Patient views scheduled appointments
 - Patient can accept or decline proposed appointments
 - System notifies therapist of patient decisions



4. For therapist search:
 - Patient enters search criteria
 - System displays matching therapists
 - Patient can view detailed therapist profiles
5. For booking:
 - Patient selects available time slot
 - System creates appointment
 - Confirmation is sent to both patient and therapist

Postconditions:

- Appointments are scheduled in the system
- Both patient and therapist calendars are updated
- Patient profile information is current
- Therapy session details are recorded

Alternative Flows:

- If no suitable therapist is found, system suggests broadening search criteria
- If selected time slot becomes unavailable, system offers alternatives
- If patient declines appointment, therapist is notified and slot is freed

Notes:

- System ensures patient privacy and data protection
- Appointment notifications are sent via multiple channels
- Therapist availability is updated in real-time
- Patient preferences are stored to improve future therapist matching

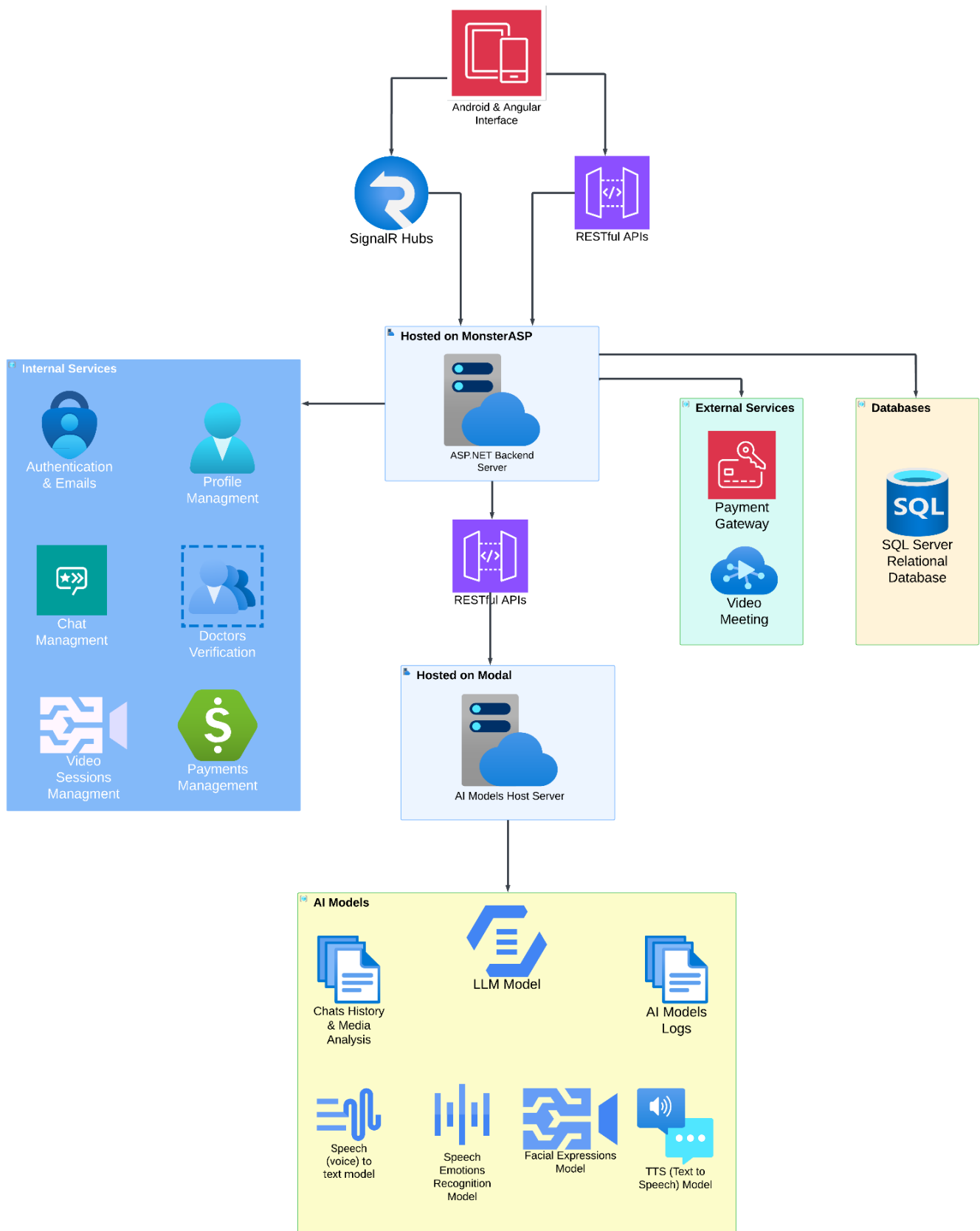


2.6 System Block Diagram

The system block diagram provides a high-level view of the core components of the platform and the data flow between them. It shows how different modules interact, including the user interface, backend services, and AI models. This helps in understanding the system architecture briefly and how the components contribute to delivering mental health services.



System Block Diagram Image



System Block Diagram Explanation

The diagram illustrates a comprehensive telemedicine system architecture with several interconnected components organized in a hierarchical structure. Here's a detailed explanation of each component and their relationships:

Frontend Layer

At the top of the diagram, the system offers a dual-interface approach:

- **Android & Angular Interface:** Provides mobile and web access for users
- Connected to backend services via two communication methods:
 - **SignalR Hubs:** Enables real-time bidirectional communication (chats)
 - **RESTful APIs:** Facilitates standard HTTP-based data exchange

Backend Services

The core of the system is built on:

- **ASP.NET Backend Server:** Hosted on MonsterASP, serving as the central processing hub
- **Communication with AI Infrastructure:** The ASP.NET server connects directly to the Modal server via RESTful APIs, enabling it to request and receive AI processing capabilities
- Connected to three main service categories:
 - **Internal Services**
A suite of modules handling user-facing functionality:
 - **Authentication & Emails:** Manages user access and communication
 - **Profile Management:** Handles user profile data
 - **Chat Management:** Facilitates text-based communication
 - **Doctors Verification:** Validates healthcare provider credentials
 - **Video Sessions Management:** Controls Video appointment Bookings
 - **Payments Management:** Processes financial transactions
 - **External Services**
Third-party integrations enhancing system capabilities:
 - **Payment Gateway:** Managing Payment from patients to doctors
 - **Video Meeting:** Provides conferencing capabilities
 - **Database Layer**
SQL Server Relational Database: Stores structured data for the entire system



AI Infrastructure

A sophisticated AI ecosystem consisting of:

- **AI Models Host Server:** Hosted on Modal, serving as middleware for AI services
- **RESTful API Interface:** Receives requests from the ASP.NET backend using FastAPI Endpoints and returns AI processing results
- **Multiple specialized AI models:**
 - LLM Model: Large Language Model for advanced text processing
 - Chat History & Media Analysis: load the history and increment with each new message
 - AI Models Logs: Tracking AI performance and usage
 - Speech (voice) to text model: Converts audio to written text
 - Speech Emotions Recognition Model: Detects emotional cues in speech
 - Facial Expressions Model: Analyzes visual emotional cues from images/videos
 - TTS (Text to Speech) Model: Converts LLM text responses to natural-sounding speech

This architecture enables a complete telemedicine experience with secure communication channels, verified healthcare providers, integrated payment processing, and AI-enhanced interactions for both patients and medical professionals. The integration between the ASP.NET backend and Modal's FastAPI AI models endpoints allows the system to leverage advanced AI capabilities while maintaining a centralized application flow.



2.7 ERD (Entity Relationship Diagram)

Introduction to ERD (Entity-Relationship Diagram)

An **Entity-Relationship Diagram (ERD)** is a visual representation of the structure of a database. It shows how data is organized and how entities (tables) relate to one another.

Purpose of an ERD:

- To illustrate the data model in a clear and structured format.
- To identify relationships and constraints among data.
- To assist in communication between developers, analysts, and stakeholders.
- To support database normalization and integrity.

Main Components of an ERD:

- **Entities:** Represented as rectangles; they correspond to database tables (e.g., Users, Chats).
- **Attributes:** Listed within the entity; these are the columns of the table.
- **Primary Keys (PK):** Uniquely identify each record in a table.
- **Foreign Keys (FK):** Establish relationships between entities.
- **Relationships:** Represented by lines connecting entities, with symbols indicating cardinality (e.g., one-to-many).

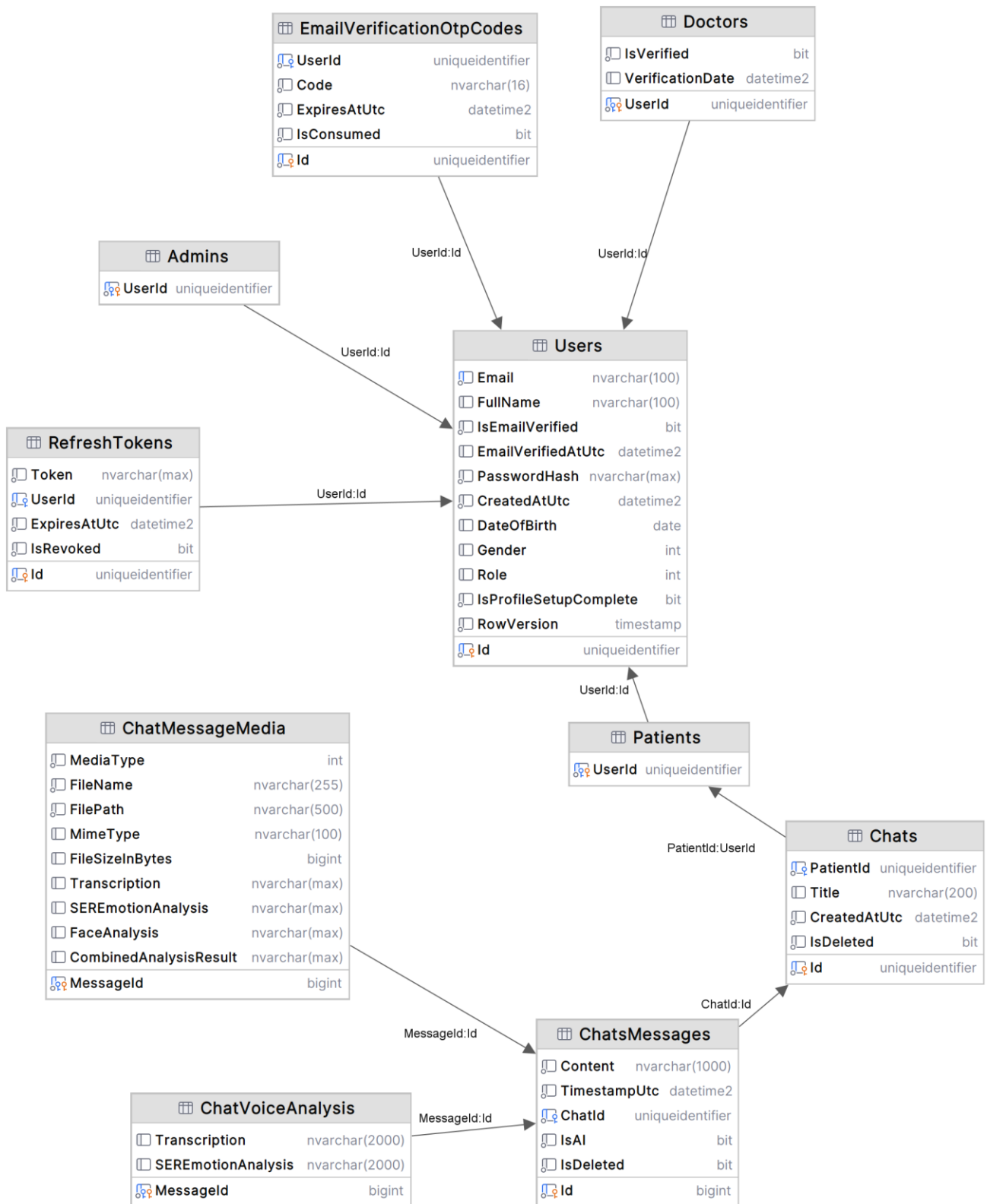
Types of Relationships:

- **One-to-One (1:1):** Each record in one table relates to one in another.
- **One-to-Many (1:N):** One record in a table relates to multiple in another.
- **Many-to-Many (M:N):** Requires a junction table to represent.

ERDs are essential in database design as they serve as blueprints for implementing and understanding the data structure.



ERD Image



Entities Descriptions

Users

Purpose: Central entity representing all users.

Column	Type	Description
Id	uniqueidentifier	Primary Key
Email	nvarchar(100)	Unique, required
FullName	nvarchar(100)	User's full name
IsEmailVerified	Bit	Email verification flag
EmailVerifiedAtUtc	datetime2	Timestamp of verification
PasswordHash	nvarchar(max)	Securely stored password hash
CreatedAtUtc	datetime2	Creation timestamp
DateOfBirth	Date	Optional
Gender	Int	Enum (e.g., 0: Unknown, 1: Male, 2: Female)
Role	Int	User's system role
IsProfileSetupComplete	Bit	Indicates if setup is finished
RowVersion	timestamp	Concurrency control

Admins / Doctors / Patients

Purpose: Role-specific user extensions.

Each table has:

- UserId as Primary Key and Foreign Key to Users.Id
- Doctors additionally includes:
 - IsVerified (bit)
 - VerificationDate (datetime2)

Relationships: One-to-one with Users.



EmailVerificationOtpCodes

Purpose: Stores OTP codes for user email validation.

Column	Type	Description
Id	uniqueidentifier	Primary Key
UserId	uniqueidentifier	FK to Users
Code	nvarchar(16)	OTP code
ExpiresAtUtc	datetime2	Expiry time
IsConsumed	Bit	Whether it was used

RefreshTokens

Purpose: Session persistence for authentication.

Column	Type	Description
Id	uniqueidentifier	Primary Key
Token	nvarchar(max)	Authentication token
UserId	uniqueidentifier	FK to Users
ExpiresAtUtc	datetime2	Expiration date/time
IsRevoked	Bit	If the token is no longer valid

Chats

Purpose: Represents chat sessions between a patient and system.

Column	Type	Description
Id	uniqueidentifier	Primary Key
PatientId	uniqueidentifier	FK to Patients
Title	nvarchar(200)	Optional session title
CreatedAtUtc	datetime2	Chat creation timestamp
IsDeleted	Bit	Soft delete flag



ChatsMessages

Purpose: Stores messages within a chat session.

Column	Type	Description
Id	bigint (identity)	Primary Key
Content	nvarchar(1000)	Message body
TimestampUtc	datetime2	Time of message
ChatId	uniqueidentifier	FK to Chats
IsAI	Bit	AI-generated or user
IsDeleted	Bit	Soft delete flag

ChatMessageMedia

Purpose: Attaches files and analysis to messages.

Column	Type	Description
MessageId	bigint	PK + FK to ChatsMessages
MediaType	Int	Enum (image/audio/video.)
FileName	nvarchar(255)	Original filename
FilePath	nvarchar(500)	Storage path
MimeType	nvarchar(100)	MIME type
FileSizeInBytes	bigint	Size in bytes
Transcription	nvarchar(max)	Speech-to-text content
SEREmotionAnalysis	nvarchar(max)	Emotion detection results
FaceAnalysis	nvarchar(max)	Visual emotion analysis
CombinedAnalysisResult	nvarchar(max)	Consolidated insights

ch



ChatVoiceAnalysis

Purpose: Stores speech emotion recognition and transcription of Voice mode.

Column	Type	Description
MessageId	Bigint	PK + FK to ChatsMessages
Transcription	nvarchar(2000)	Text derived from audio
SEREmotionAnalysis	nvarchar(2000)	Emotion classification

Relationships Overview

Parent Table	Child Table	Type	Notes
Users	Admins, Doctors, Patients	One-to-One	Role-specific extensions
Users	EmailVerificationOtpCodes	One-to-Many	Email validation records
Users	RefreshTokens	One-to-Many	Auth token storage
Patients	Chats	One-to-Many	Each patient has chats
Chats	ChatsMessages	One-to-Many	Message history
ChatsMessages	ChatMessageMedia	One-to-One	Optional media attachments
ChatsMessages	ChatVoiceAnalysis	One-to-One	Optional voice analysis

Notes

- The database supports soft deletes via IsDeleted flags. as will be supported in the system in the future
- AI interaction is traceable via IsAI, SEREmotionAnalysis, FaceAnalysis, and Transcription.



2.8 Tools & Technology used

2.8.1 Mobile Application

- **Jetpack Compose**

Jetpack Compose streamlines UI development in Android by offering a modern, declarative approach to building user interfaces. It allows developers to create dynamic and responsive layouts using Kotlin code, eliminating the need for XML. This enhances code readability, promotes reusable components, and simplifies UI updates, ultimately leading to more efficient and maintainable Android applications.



- **Ktor**

Ktor simplifies the development of asynchronous servers and clients in Kotlin by providing a flexible and lightweight framework. It allows developers to build RESTful APIs, web applications, and network clients using a concise DSL. With support for coroutines, Ktor ensures non-blocking operations, enhancing performance and scalability while keeping the code clean, readable, and easy to maintain.



- **Koin**

Koin plays a vital role in managing dependencies in Kotlin applications by offering a simple and lightweight dependency injection framework. It eliminates the need for boilerplate code and complex configurations, allowing developers to define and inject dependencies using a clear and concise DSL. This improves code modularity, testability, and maintainability, making it easier to scale and manage Kotlin-based projects.



2.8.2 Web Application

- **Angular**

Angular is a modern, open-source web application framework developed by Google, used for building dynamic and single-page web applications. It is based on TypeScript and follows a component-based architecture, which helps in creating scalable and maintainable applications. Angular provides powerful built-in features such as two-way data binding, dependency injection, and routing, which streamline the development process and improve performance. Its structured approach allows developers to build complex user interfaces with ease, making it a preferred choice for developing responsive and interactive web applications.



- **Bootstrap**

Bootstrap is a popular front-end framework developed by Twitter that simplifies the design of responsive and mobile-first websites. It offers a wide range of pre-designed components such as buttons, forms, modals, and navigation bars, along with a powerful grid system that adapts seamlessly to different screen sizes. By using Bootstrap, developers can create visually appealing and consistent user interfaces quickly without writing extensive custom CSS. Its compatibility with frameworks like Angular makes it an excellent tool for building modern, clean, and user-friendly web applications.



2.8.3 Back-end

- **ASP.NET C#**

A powerful, cross-platform framework for building modern web APIs. In Seravian, it's used to expose secure HTTP endpoints for registration, login, profile management, chat, and media uploads.



- **SignalR**



A real-time library in ASP.NET Core that enables persistent connections between client and server over WebSockets or fallback transports. Used in Seravian to support live chat between users and the AI assistant.



- **JWT (JSON Web Tokens)**

Compact and secure tokens used to authenticate and authorize users. JWTs are issued on login and validated on every request, including SignalR connections (via query string or headers).



- **MailKit**

MailKit is a modern, fully-featured email-sending library for .NET. In your project Seravian, it's used to send secure OTP codes during email verification.

mailkit



- **Microsoft SQL Server**

The primary relational database used to store all structured data: users, chat messages, roles, refresh tokens, OTPs, and chat metadata.



- **Entity Framework Core**

EF Core is used as the ORM for querying and updating the database using C#. It translates LINQ queries to SQL and handles relationship mapping, tracking, and migrations.



2.8.4 AI

- **Model Hosting & Serving**
 - Hugging Face Transformers – Pretrained models hosted or downloaded from the Hugging Face Hub.
 - GitHub – Source of various open-source machine learning models integrated into your system.
 - Modal – Used for deploying and scaling model inference endpoints as APIs.
- **API & Backend**
 - FastAPI – For building high-performance Python APIs to serve models.
 - Uvicorn – ASGI server used with FastAPI for running the API endpoints.
- **Containerization & Deployment**
 - Modal CLI / Modal SDK – For writing deployment logic, defining functions, and managing infrastructure as code.
- **Data Handling**
 - Pandas / NumPy – For data processing.
 - PIL / OpenCV – For image handling.
- **Model Integration**
 - Transformers / SentenceTransformers /etc.
 - DeepFace / torchvision / Tortoise /Mentallama /Etc.
- **Others**
 - Pydantic – Used within FastAPI for data validation and serialization.
 - Requests / HTTPx – For internal API calls if endpoints talk to each other.



CHAPTER THREE:

Implementation and results



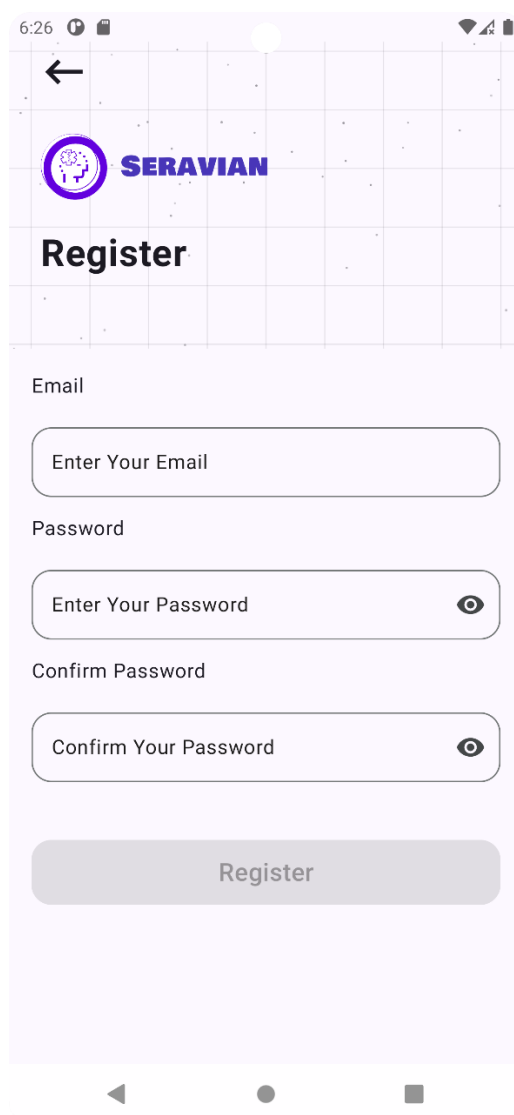
3.1. Front-End Implementation

3.1.1. Android Application

User Flow and App Overview

3.1.1.1. Sign Up

- The user must sign up when launching the app for the first time.
- The password must contain number, uppercase letter and special character and consists of minimum 8 letters. (ex.. Ser@vian12)



6:26

←

 **SERAVIAN**

Register

Email

Enter Your Email

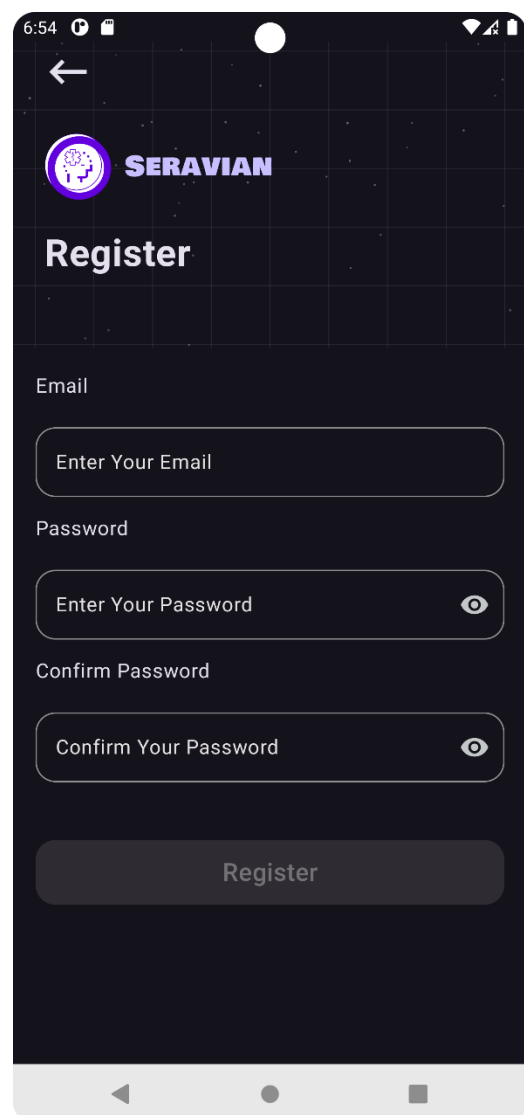
Password

Enter Your Password 

Confirm Password

Confirm Your Password 

Register



6:54

←

 **SERAVIAN**

Register

Email

Enter Your Email

Password

Enter Your Password 

Confirm Password

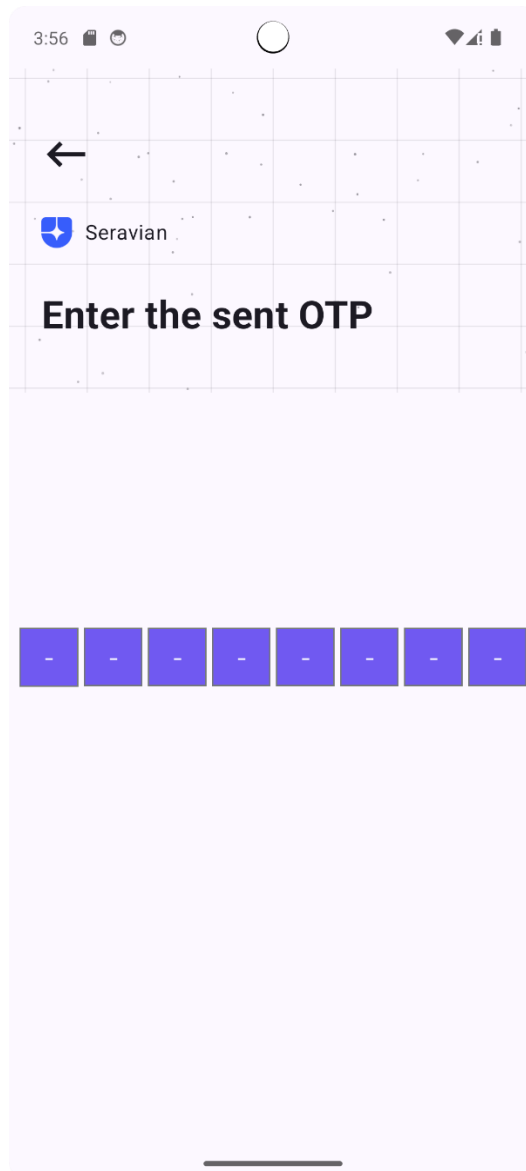
Confirm Your Password 

Register



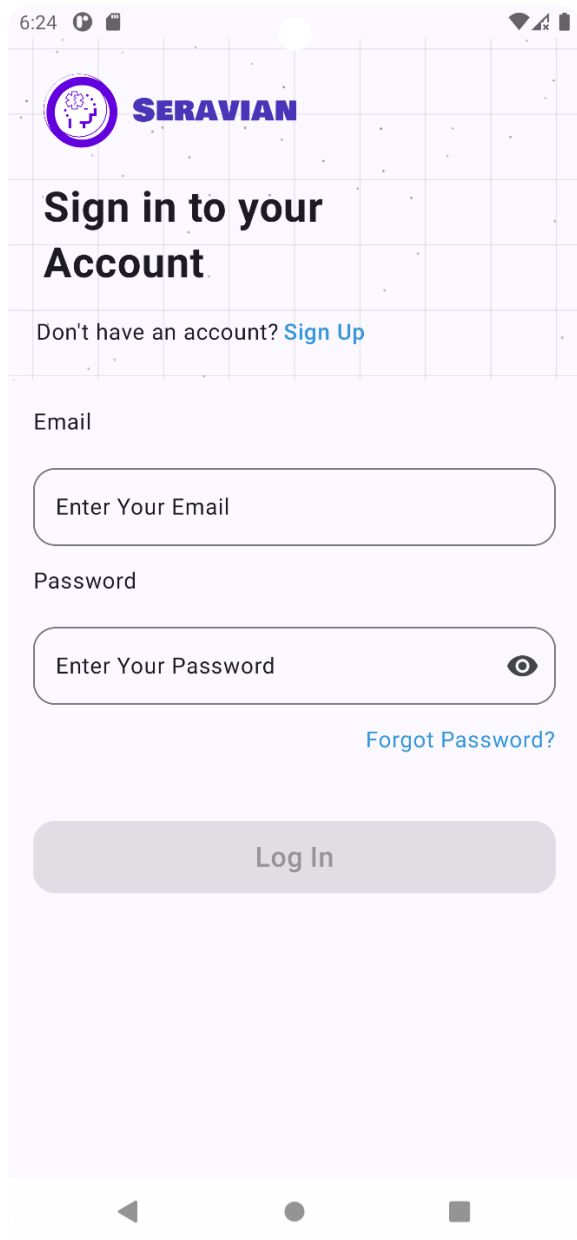
3.1.1.2. OTP Verification

- After signing up, the user is navigated to the OTP screen.
- An OTP (One-Time Password) is sent to the user's email.
- Upon successful verification, the user proceeds to the login screen.



3.1.1.3. Login

- The user logs in using their credentials.
- If it's the user's first login, they will be required to fill in personal information.



6:24

 **SERAVIAN**

Sign in to your Account

Don't have an account? [Sign Up](#)

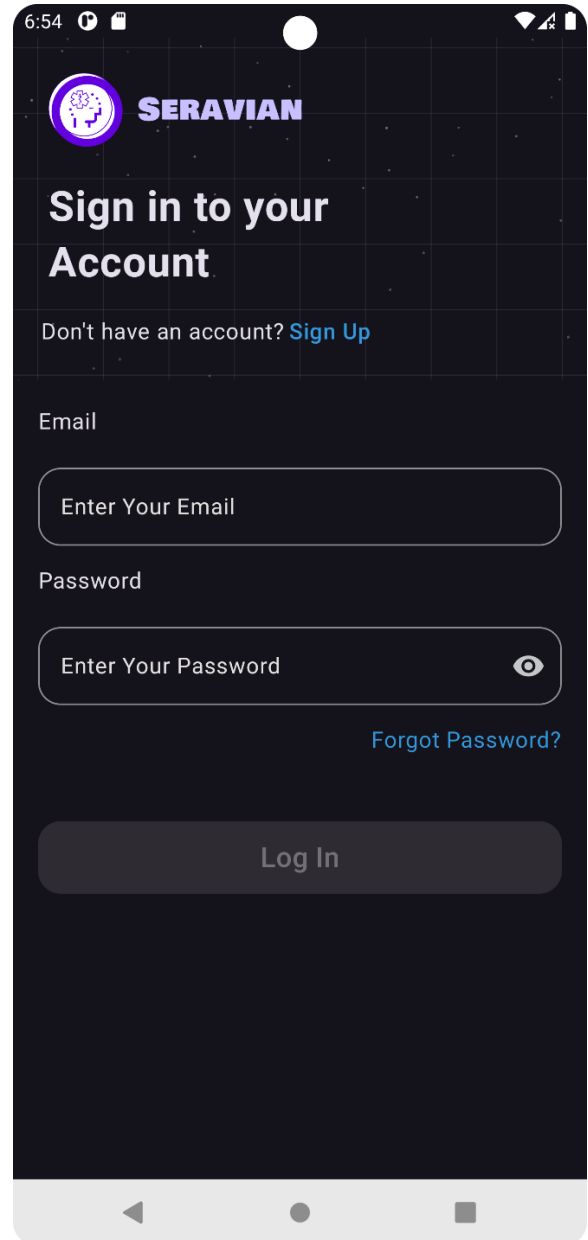
Email

Password

[Forgot Password?](#)

Log In



6:54

 **SERAVIAN**

Sign in to your Account

Don't have an account? [Sign Up](#)

Email

Password

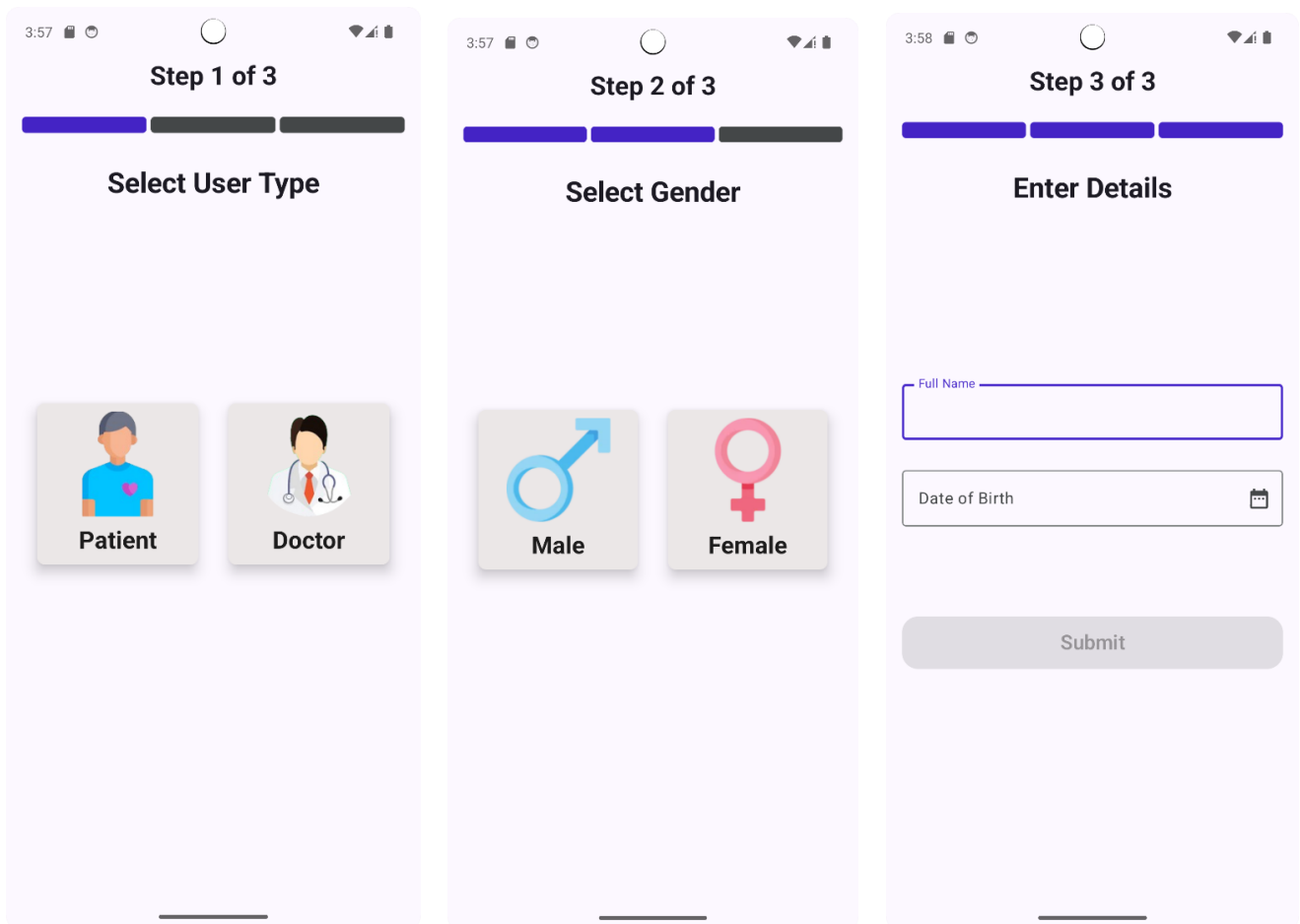
[Forgot Password?](#)

Log In



3.1.1.4. Profile Setup (First-Time Login Only)

- The user must enter the following information:
 - Role (e.g., doctor or patient)
 - Gender
 - Full name
 - Birth date
- After completing this step, the user is redirected to the home screen.



The image displays three sequential mobile app screens for profile setup, each with a status bar at the top showing time and connectivity icons.

Step 1 of 3: Select User Type

This screen features a progress bar with the first segment highlighted in blue. Below the title, there are two selectable options: "Patient" (represented by a blue shirt icon) and "Doctor" (represented by a white lab coat icon).

Step 2 of 3: Select Gender

This screen features a progress bar with the second segment highlighted in blue. Below the title, there are two selectable options: "Male" (represented by a blue male symbol icon) and "Female" (represented by a pink female symbol icon).

Step 3 of 3: Enter Details

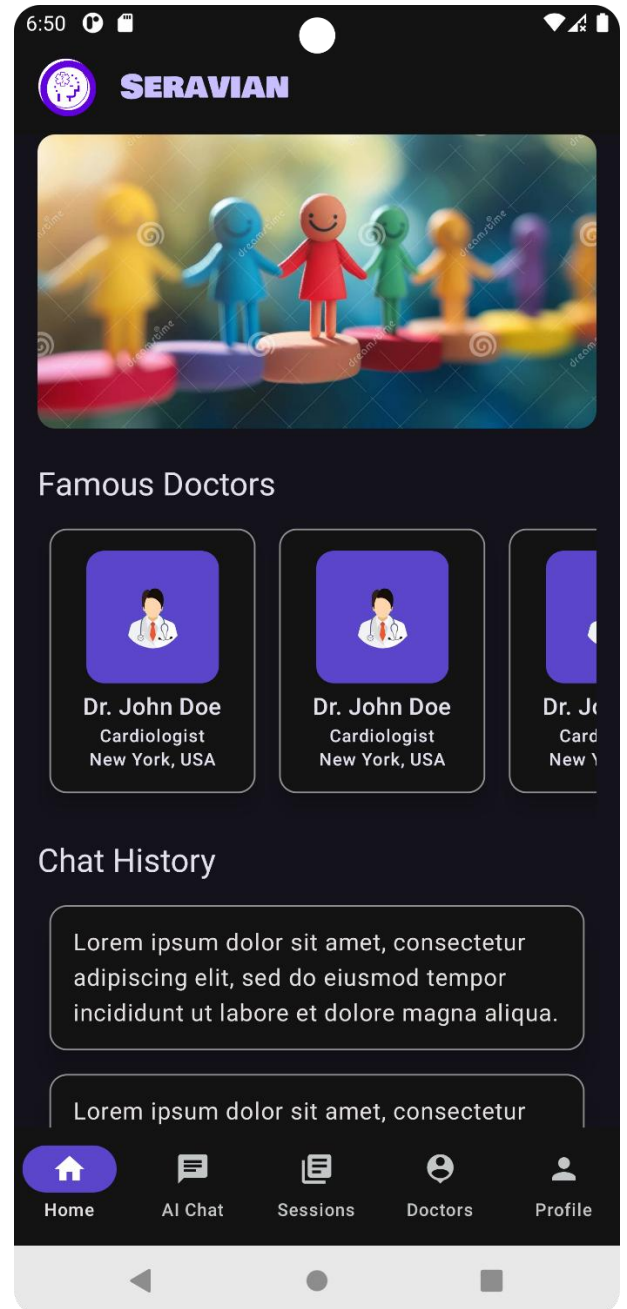
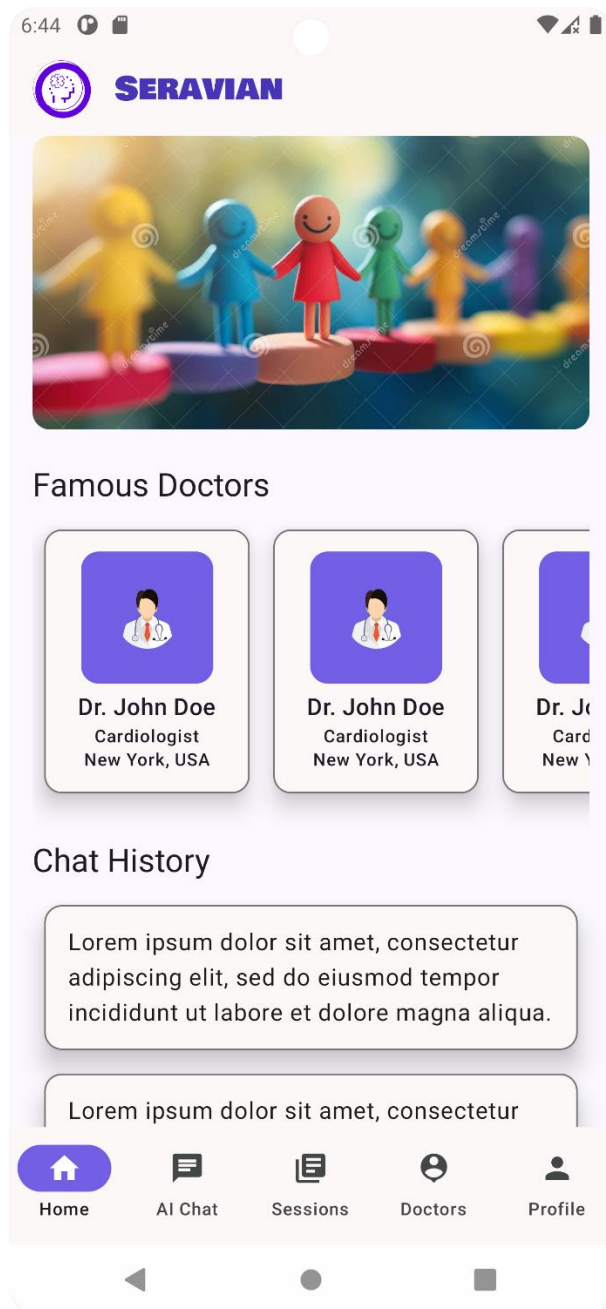
This screen features a progress bar with the third segment highlighted in blue. Below the title, there are two input fields: "Full Name" (a text input field) and "Date of Birth" (a date picker icon). A "Submit" button is located at the bottom of the screen.

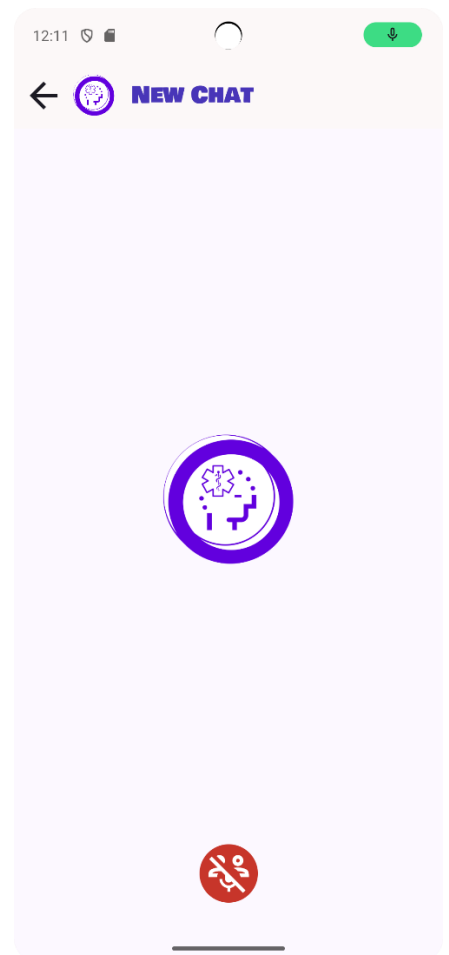
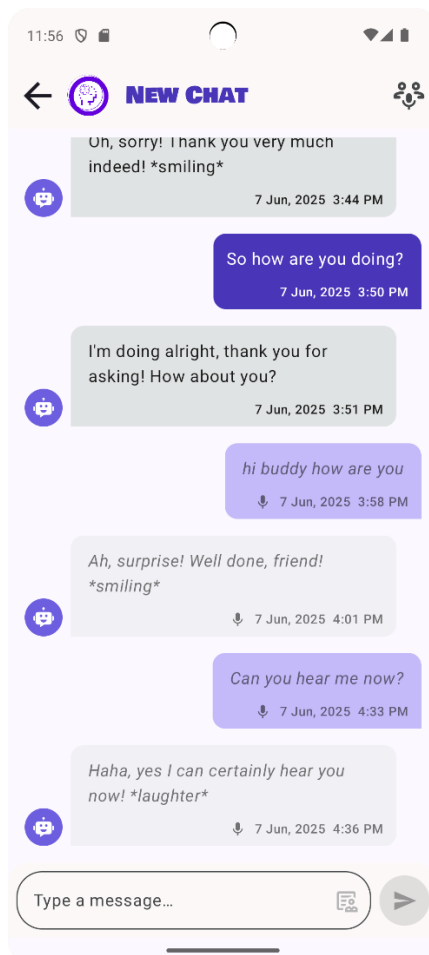
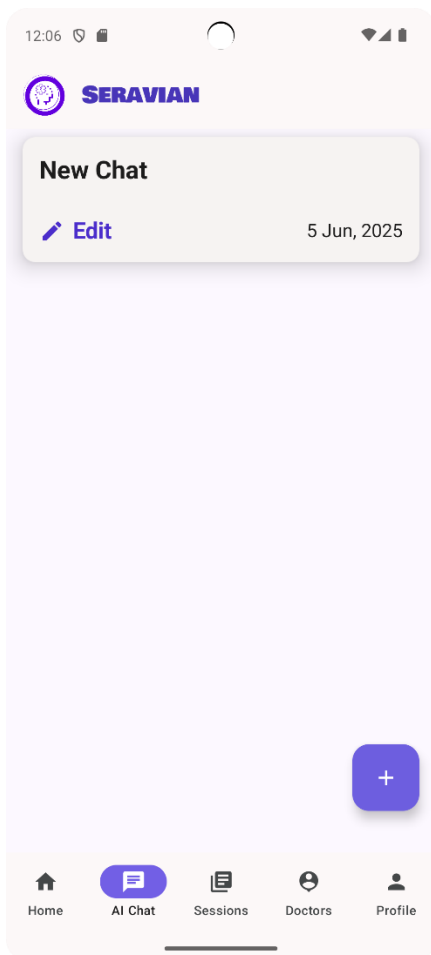


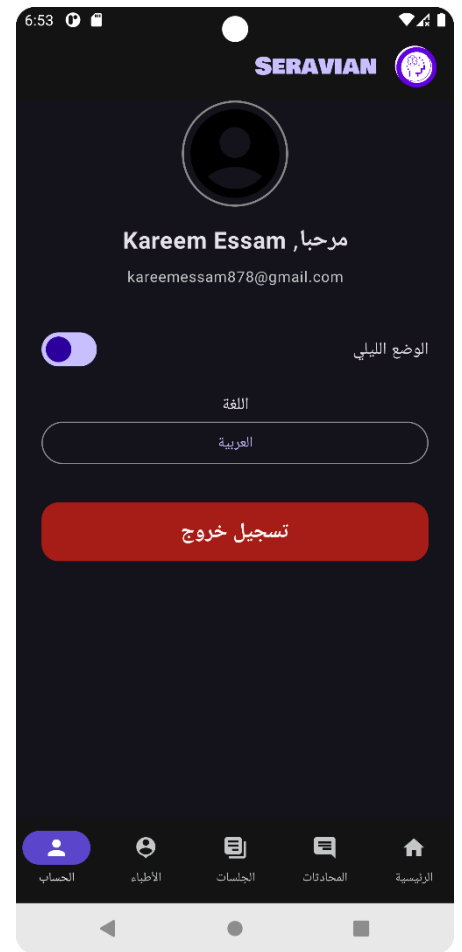
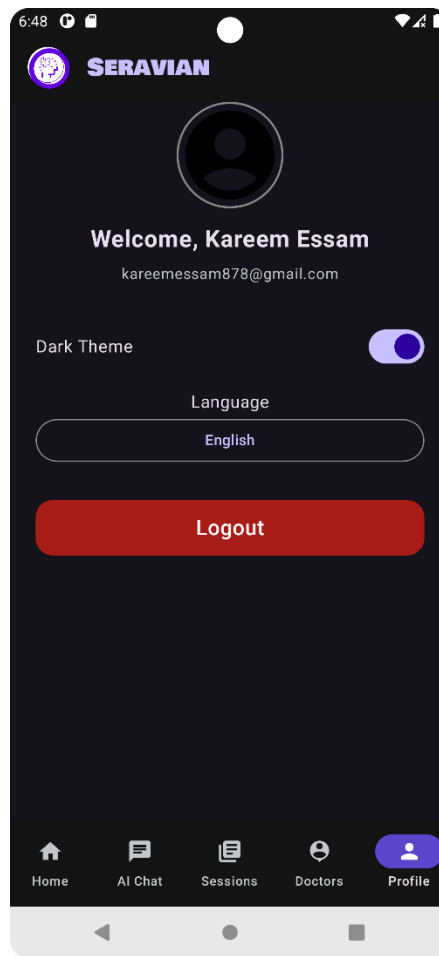
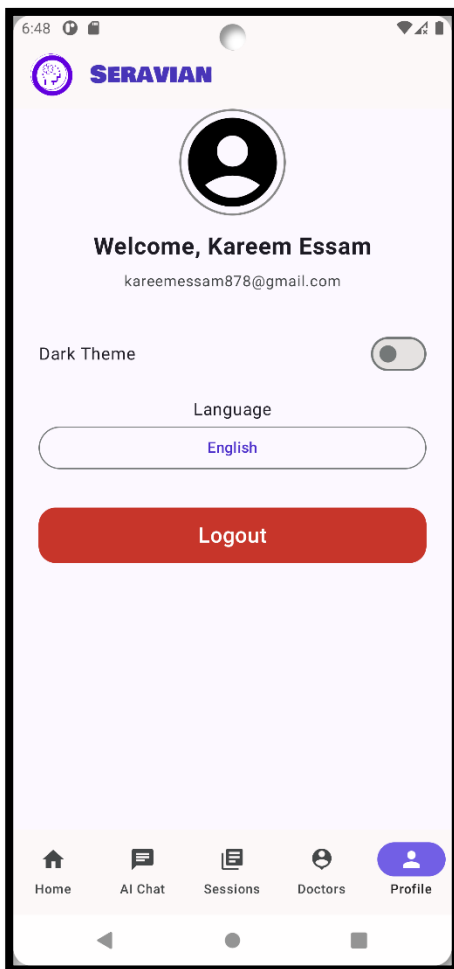
3.1.1.5 Main Structure App

The app contains 5 primary screens accessible via the bottom navigation bar:

- Home
- AI Chat
- Sessions (Future work)
- Doctors (Future work)
- Profile



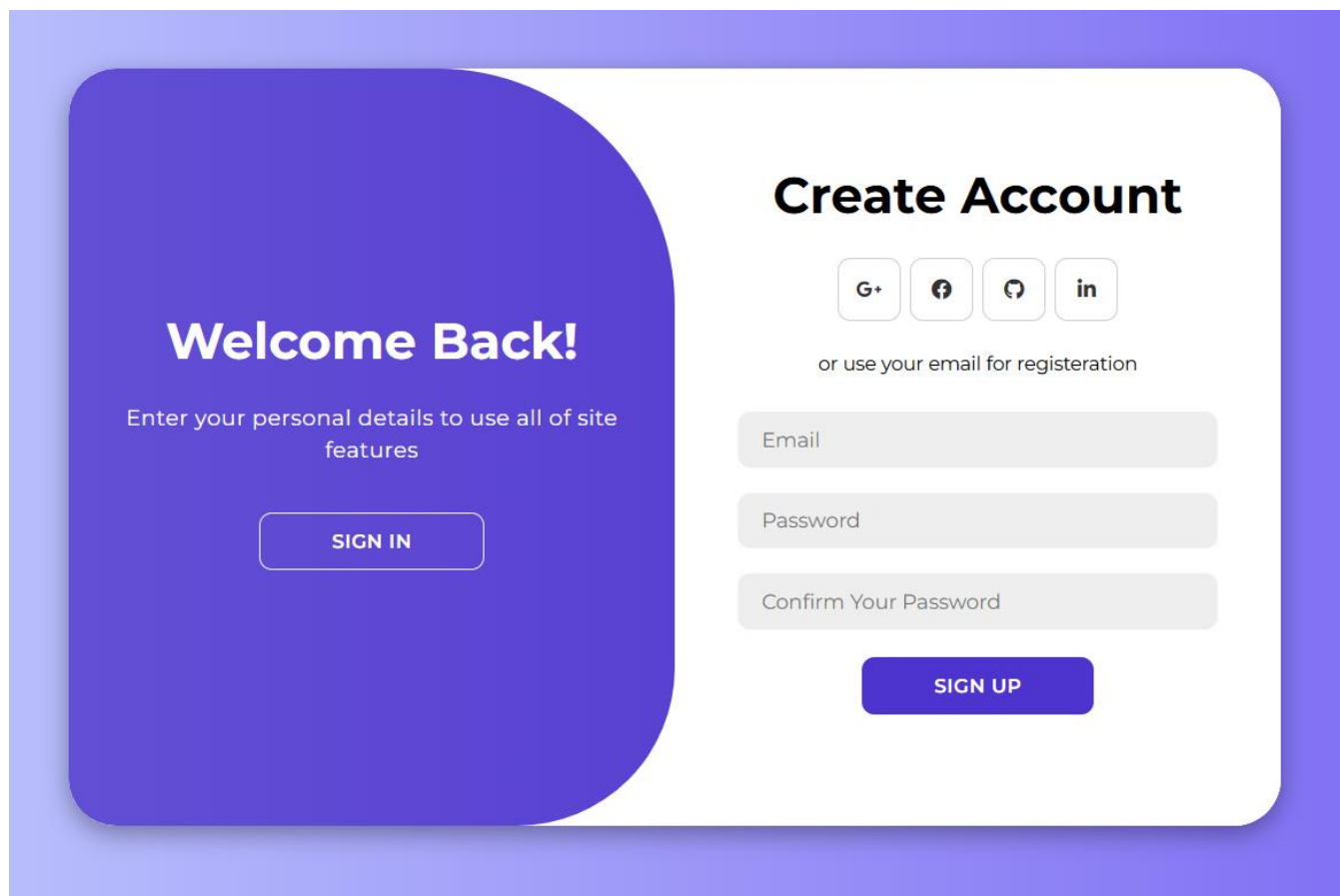




3.1.2. Web Application

3.1.2.1 Sign Up

If you're new to the app, you can create an account by tapping the "Sign Up" button on the login screen. This will guide you through a simple registration process.



Create a New Account

On the Sign Up screen, you'll need to fill in the following details:

Email: Enter a valid email address. This will be linked to your account.

Password: Create a strong password.

Confirm Password: Re-enter your password to make sure it matches.



3.1.2.2 OTP Verification

After submitting the form:

You will be directed to “OTP” page to verify your email

Check your inbox

Enter the verification code we just sent to
aboodyalhrery@gmail.com

code

Please enter the code

CONTINUE

[Resend email](#)

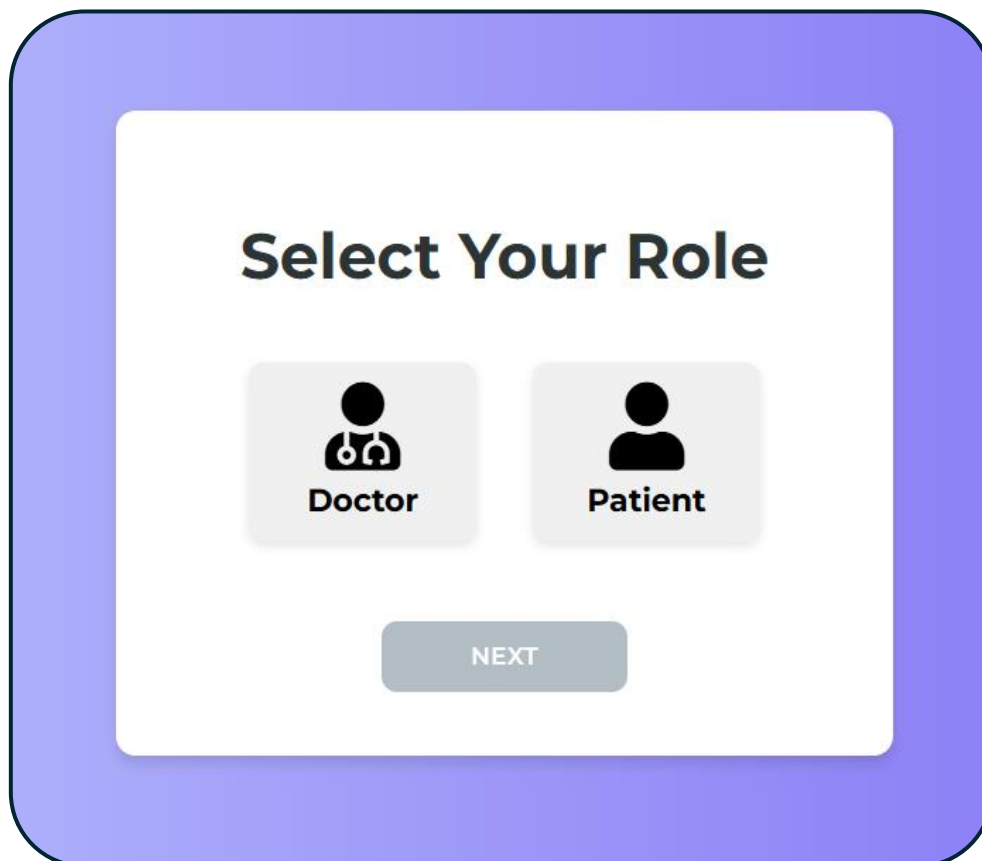


3.1.2.3 Profile Setup

After **verifying your email**:

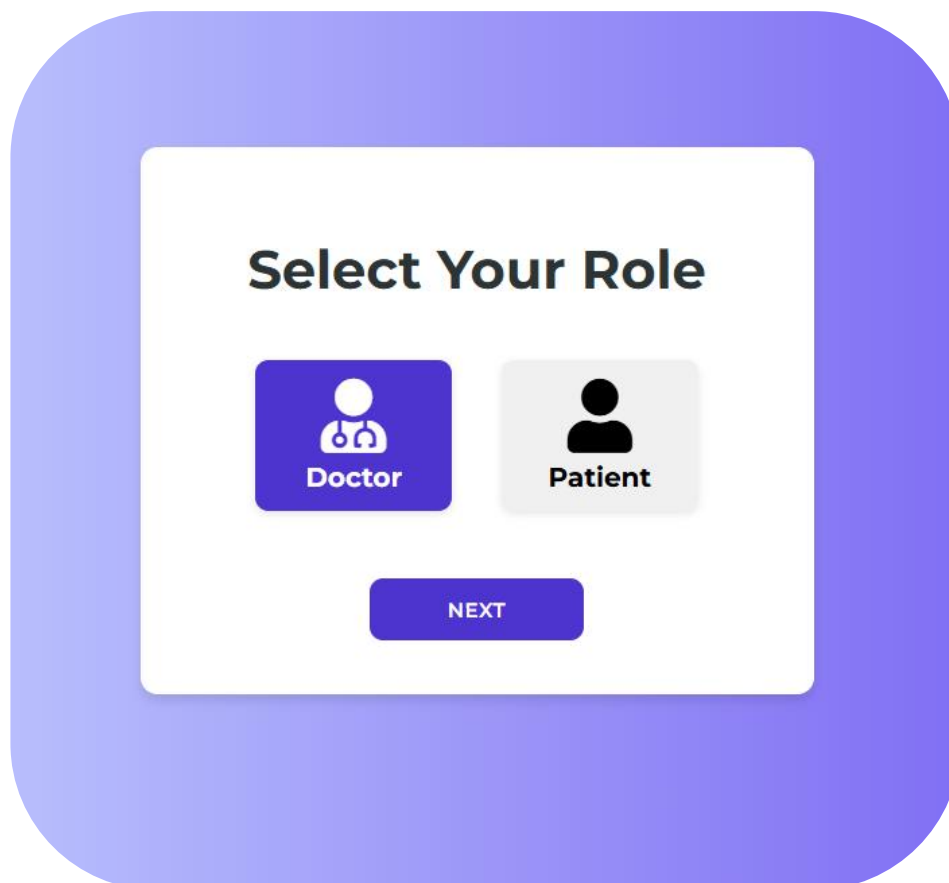
You'll be taken back to the **login** screen to **sign in**.

Once signed in, you'll continue the registration process by selecting whether you are a **doctor** or a **patient**, and then entering the required information to complete your profile.



Doctor Experience & System Interaction Guide:

if He/She choose “Doctor” as the role then after filling the information on the “User Information” form they will be directed to another form to complete doctor’s related information, after that (Future Work) the admin will verify his/her “License” and then the doctor can proceed to “Doctor Dashboard”.



User Information

Full Name

Abdalrhman

Date of Birth

07/16/2003



Gender



Male



Female



I have read and agree to Seravian's [Terms of Service](#) and [Privacy Policy](#).

CONTINUE

BACK

Doctor Verification Requests

Request Id	Date & Time	Status	Action
------------	-------------	--------	--------

ADD REQUEST



Doctor Request Form

Title

Title*

Description

Describe your proficiency in your field of expertise

Session Price (in EGP)

Enter session price

Attachments (upto 10 files & max size of 15MBs)

Choose File No file chosen

Choose File No file chosen

ADD ATTACHMENT

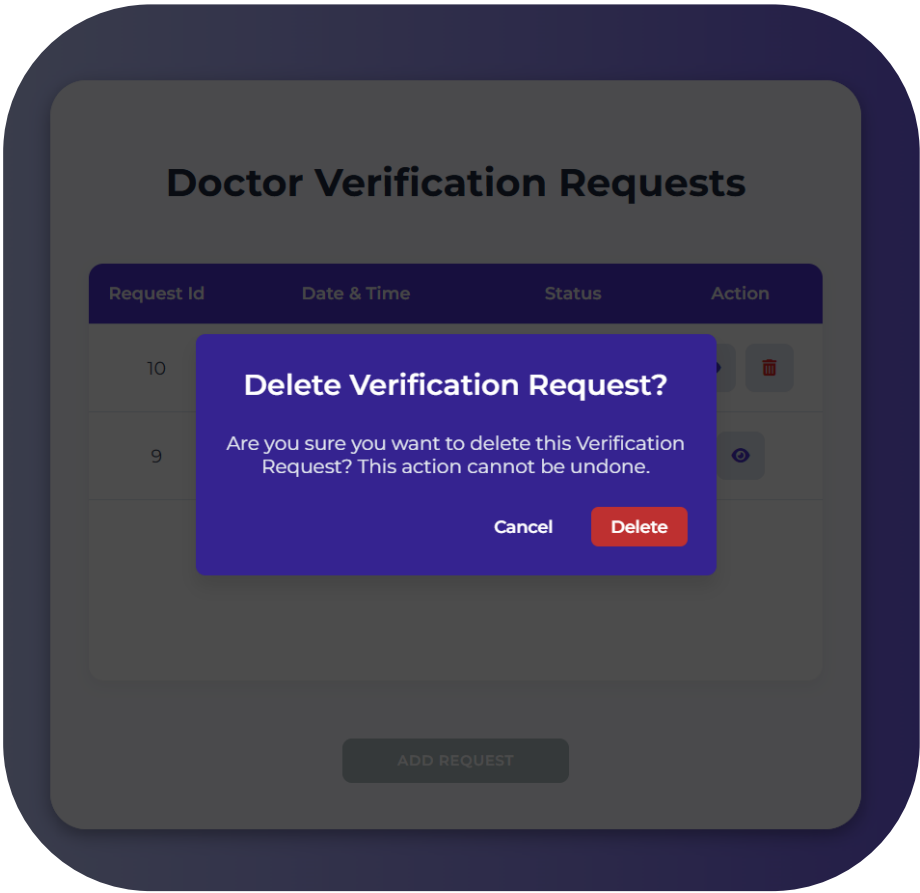
SUBMIT

Doctor Verification Requests

Request Id	Date & Time	Status	Action
10	Jun 12, 2025, 1:13:19 AM	PENDING	 
9	Jun 12, 2025, 1:12:43 AM	DELETED	

ADD REQUEST





 **Ahmed Hassan** Doctor

Dashboard

Appointments

Profile

Seravian Logout

\$80

Earnings

4

Appointments

2

Patients

Latest Bookings

 **Avinash Kr**
Booking on Oct 5, 2024 Pending

 **GreatStack**
Booking on Sep 26, 2024 Cancelled

 **GreatStack**
Booking on Sep 25, 2024 Completed

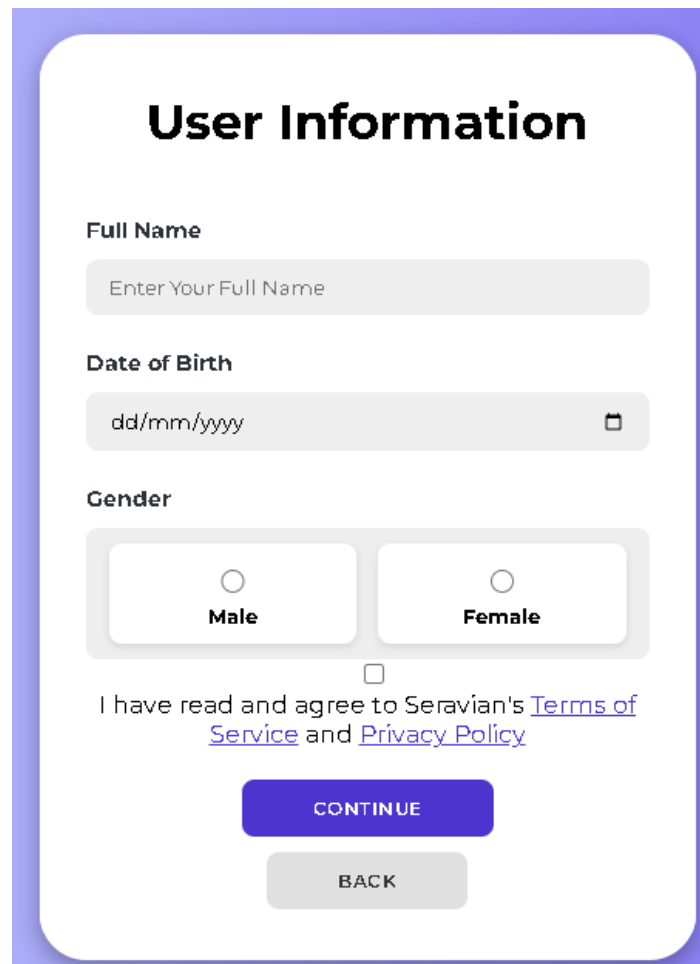
 **GreatStack**
Booking on Sep 23, 2024 Completed

SERAVIAN

55

Patient Journey & Platform Usage Guide:

Here if the user choose “Patient” as the role then after he/she fills the information on the “User Information” form he will be directed to “User Dashboard”.

A screenshot of a mobile application form titled "User Information". The form is white with rounded corners and a purple border. It contains three input fields: "Full Name" with a placeholder "Enter Your Full Name", "Date of Birth" with a placeholder "dd/mm/yyyy" and a calendar icon, and "Gender" with two radio button options: "Male" and "Female". Below the gender options is a checkbox and a line of text: "I have read and agree to Seravian's [Terms of Service](#) and [Privacy Policy](#)". At the bottom are two buttons: a purple "CONTINUE" button and a grey "BACK" button.

User Information

Full Name

Enter Your Full Name

Date of Birth

dd/mm/yyyy

Gender

☐ Male ☐ Female

☐

I have read and agree to Seravian's [Terms of Service](#) and [Privacy Policy](#)

CONTINUE

BACK

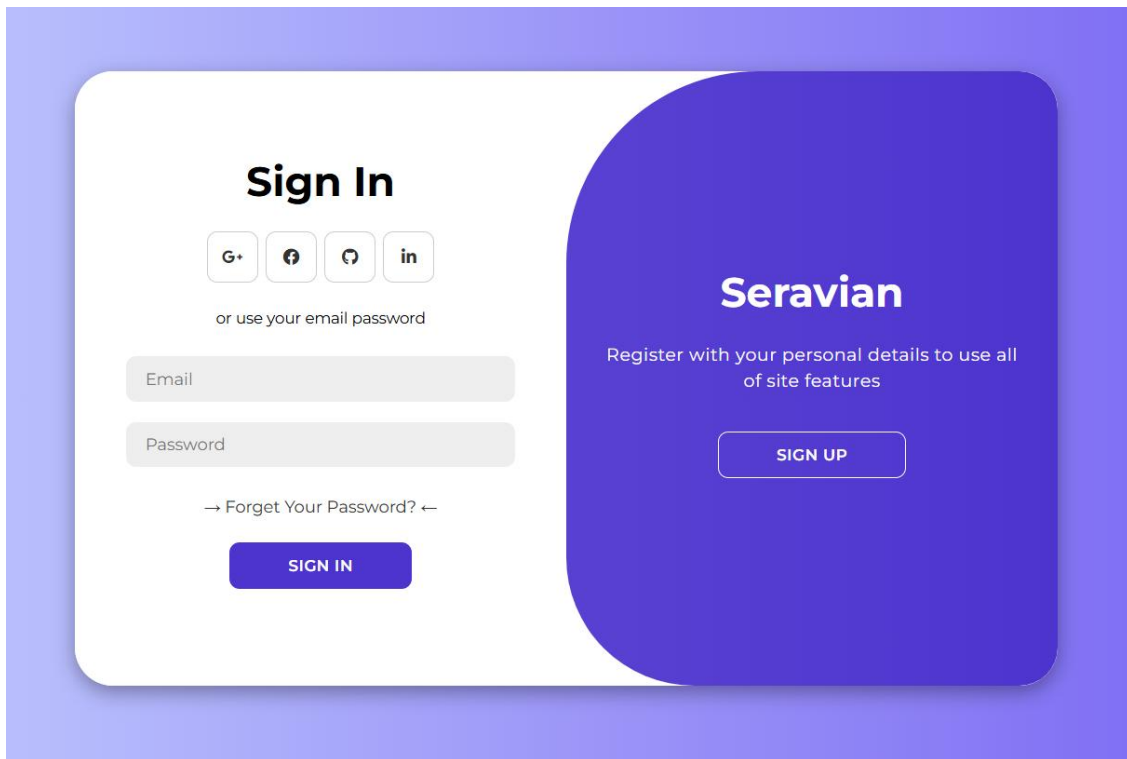


3.1.2.4 Login

Use this screen to sign in to your account and access all the features of the app.

Username: Enter the username you created when signing up. This identifies your account.

Password: Enter your secure password. Keep it private to protect your account.



The image shows the 'Sign In' screen of the Seravian app. The screen has a light blue background. On the left, there is a white rounded rectangle containing the 'Sign In' title, social media login buttons (G+, Facebook, Twitter, LinkedIn), a link to 'or use your email password', email and password input fields, a 'Forgot Your Password?' link, and a 'SIGN IN' button. On the right, there is a dark blue rounded rectangle with the 'Seravian' logo, a description of the app, and a 'SIGN UP' button.

Sign In

G+ Facebook Twitter LinkedIn

or use your email password

Email

Password

→ Forget Your Password? ←

SIGN IN

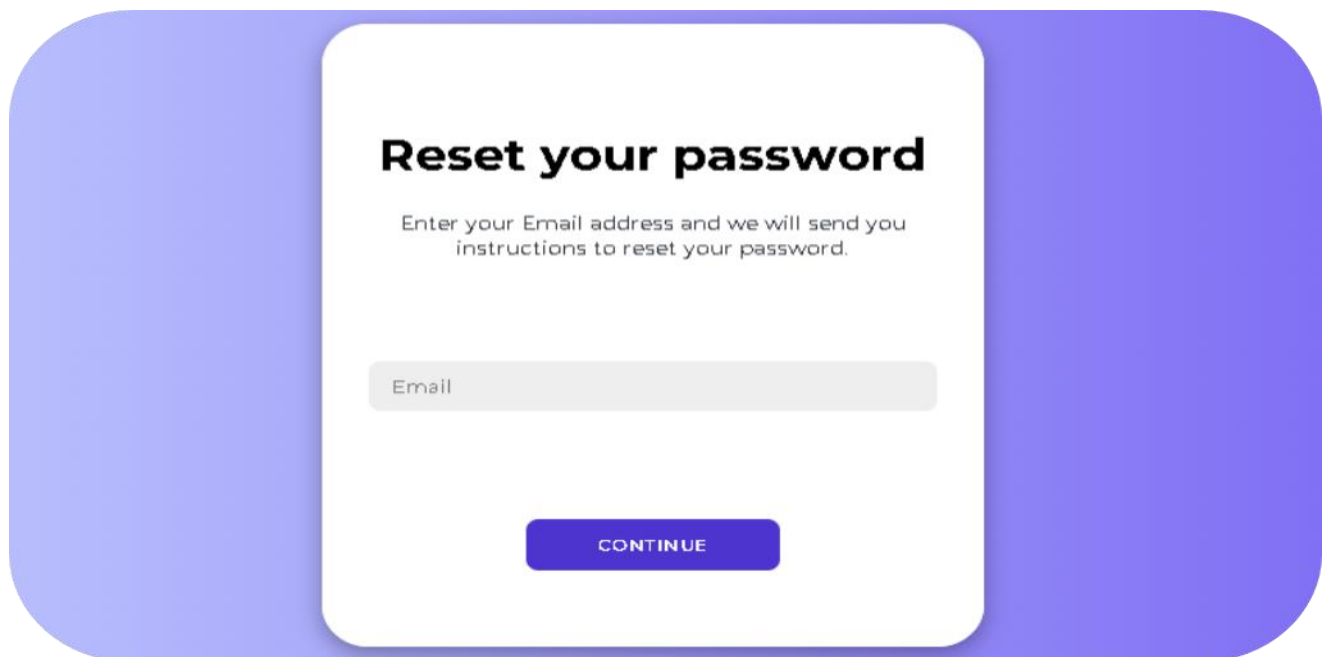
Seravian

Register with your personal details to use all of site features

SIGN UP

Forgot Your Password?

If you've forgotten your password, tap the "Forgot Password" link (usually found below the login button). You'll be guided through steps to reset and recover your account access.



The image shows the 'Reset your password' screen of the Seravian app. The screen has a light blue background. In the center, there is a white rounded rectangle containing the 'Reset your password' title, a description of the process, an email input field, and a 'CONTINUE' button.

Reset your password

Enter your Email address and we will send you instructions to reset your password.

Email

CONTINUE



Check your inbox

Enter the verification code we just sent to

code

CONTINUE

[Resend email](#)

Create new password

Password

Confirm Password

CONTINUE



3.1.2.5 Main Content:

- Home
- Profile
- AI Chat

Home:

After signing in, you'll land on the Home page. From here, you can:

Install the Mobile App: Download our mobile app for easier access to appointments, chat, and services on the go.

View Comments and Testimonials: Read feedback from other users to learn about their experiences and gain insights into the services provided on our platform.

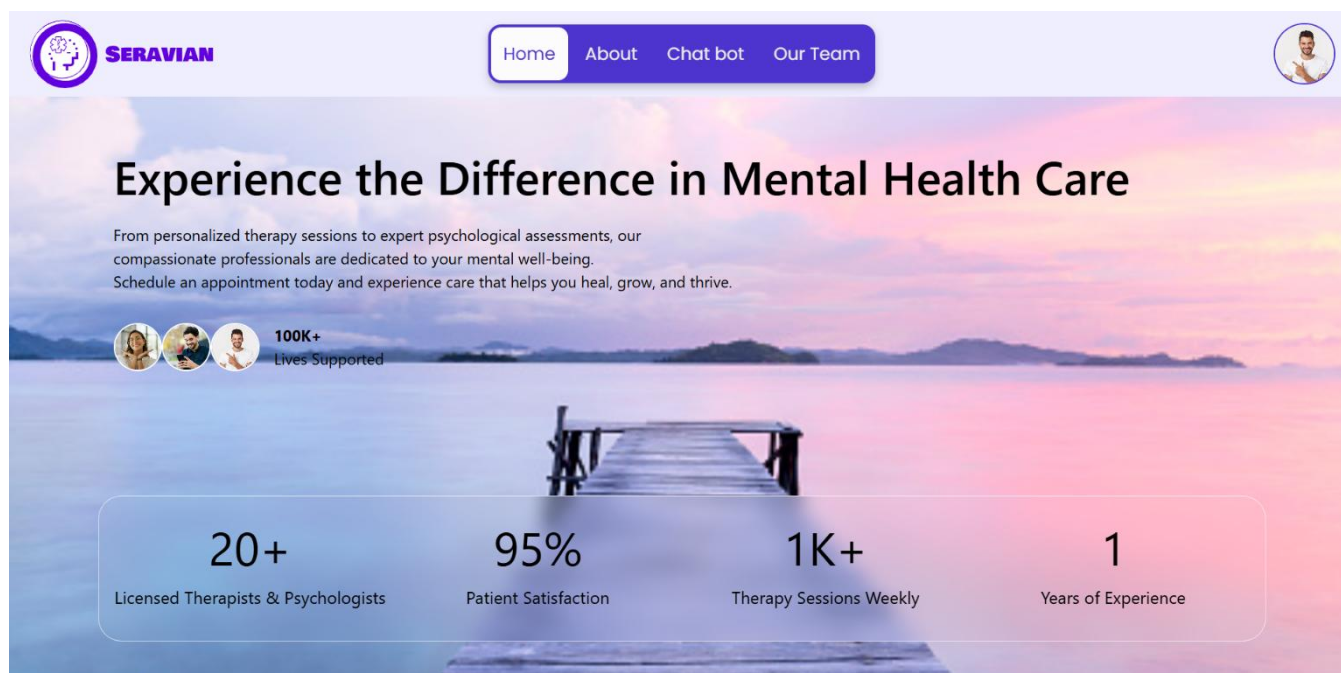
Browse Psychiatrists and Psychologists: View a list of available mental health professionals with their specialties and profiles.

Book Appointments: Choose a psychiatrist or psychologist and schedule a session at a time that works for you.

Chat with the AI Bot: Get quick answers, mental health support, or general guidance through our intelligent chatbot

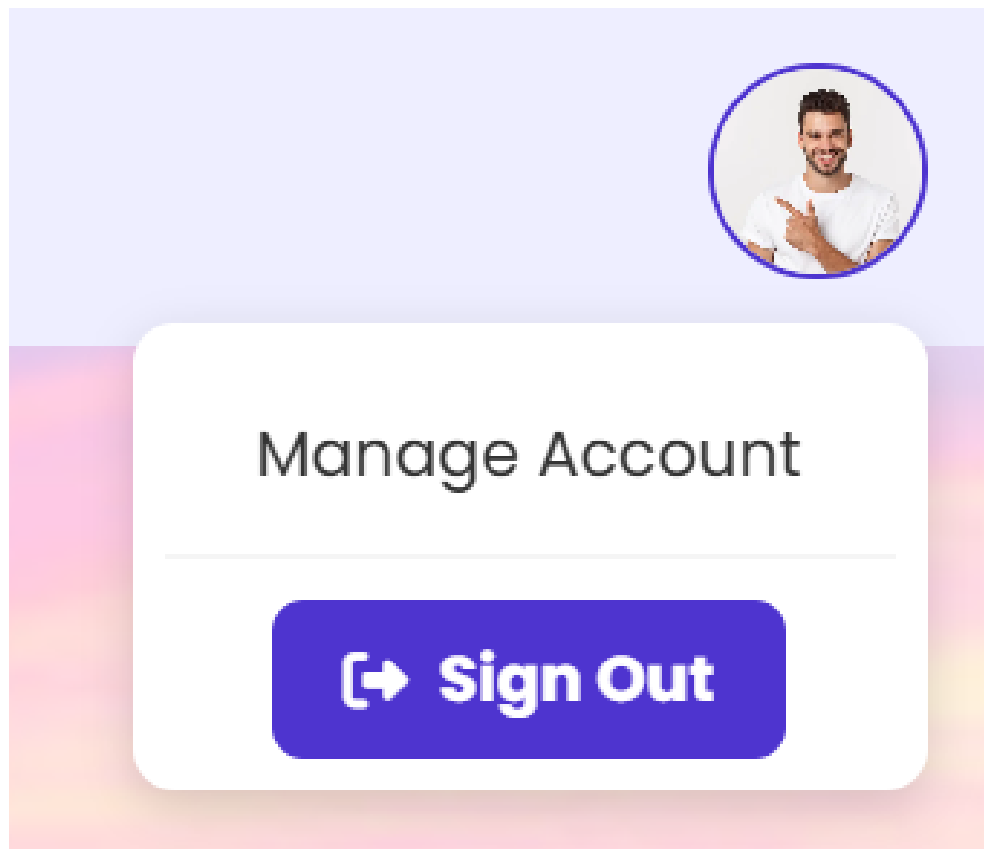
Manage Your Account: Edit your personal information and manage account settings.

The Home page is your starting point to easily access the mental health support you need.



Manage Account (Profile):

This is where you can access your account's personal data and settings :



These are the different tabs to manage your account:



John Doe

Profile

Account

Password

Profile Information

Full Name

Email

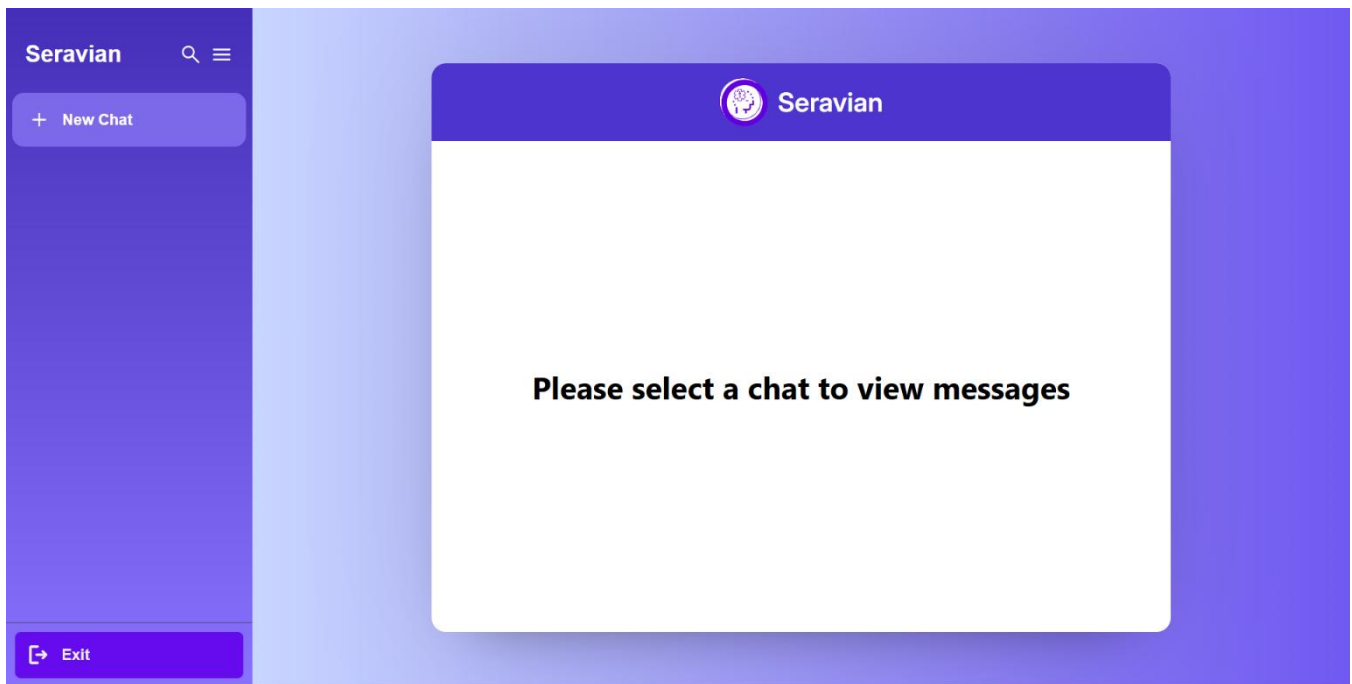
Phone Number

Save Changes

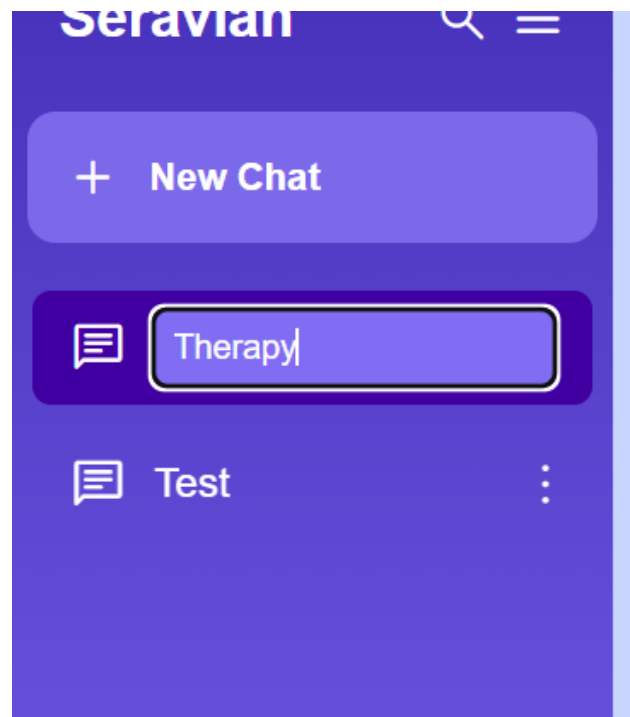
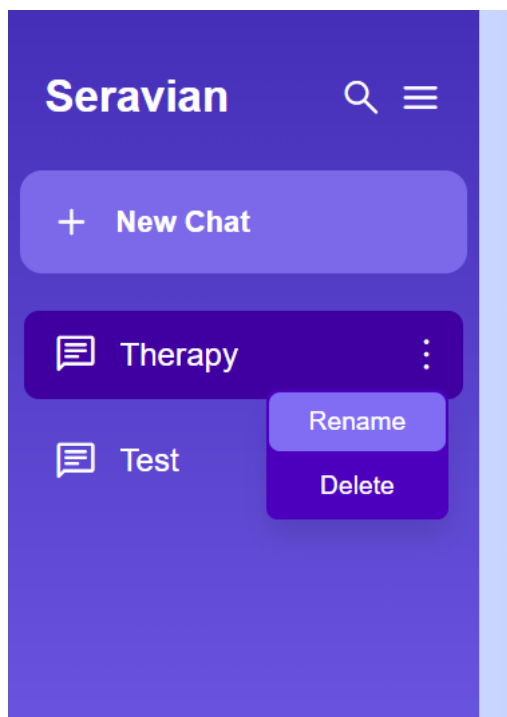


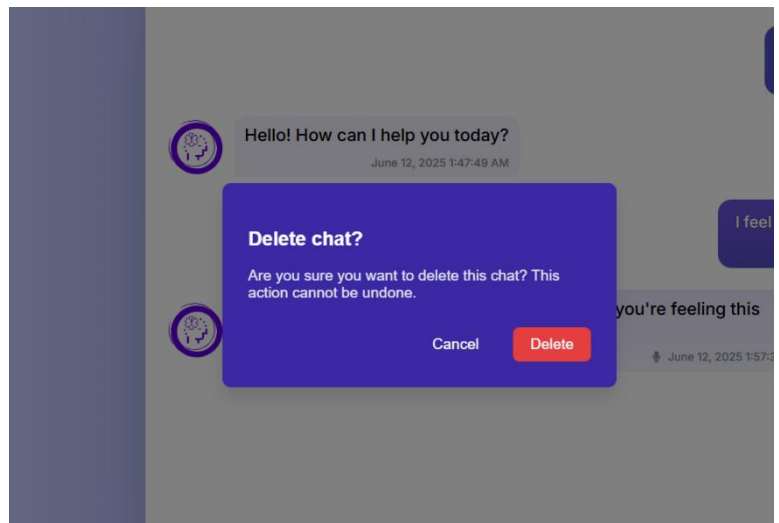
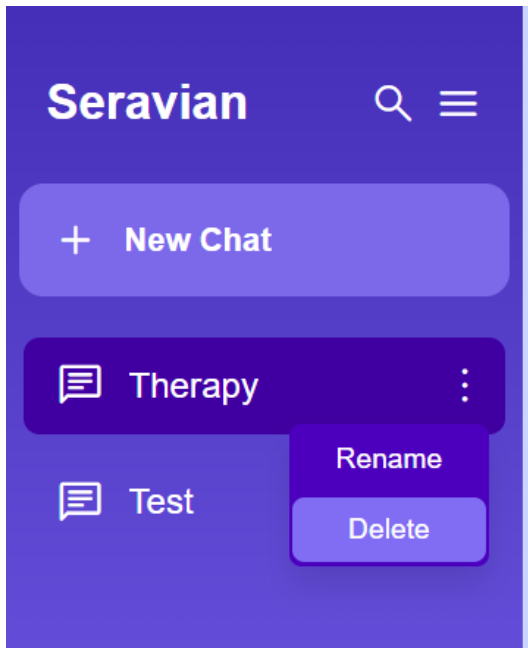
AI Chat:

This is where you can chat with the AI model through text or voice

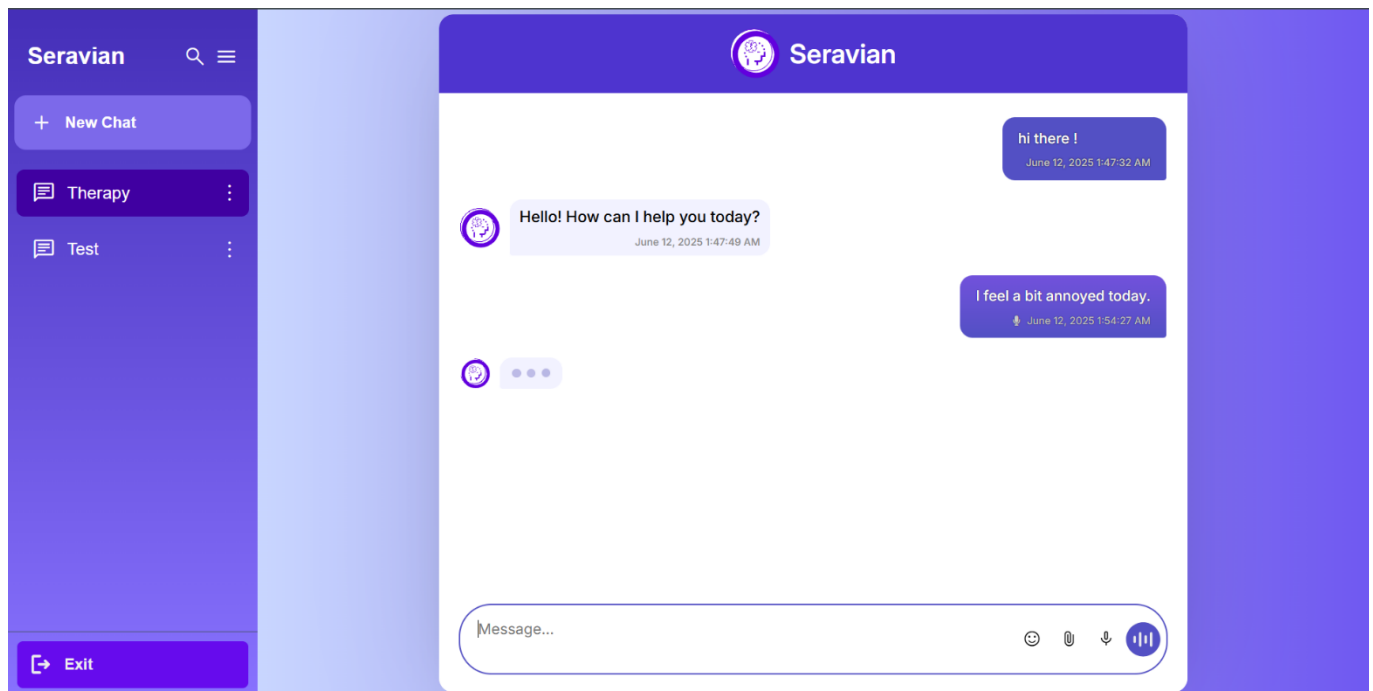


You can **create**, **update** or **delete** a chat through the side-bar controls:

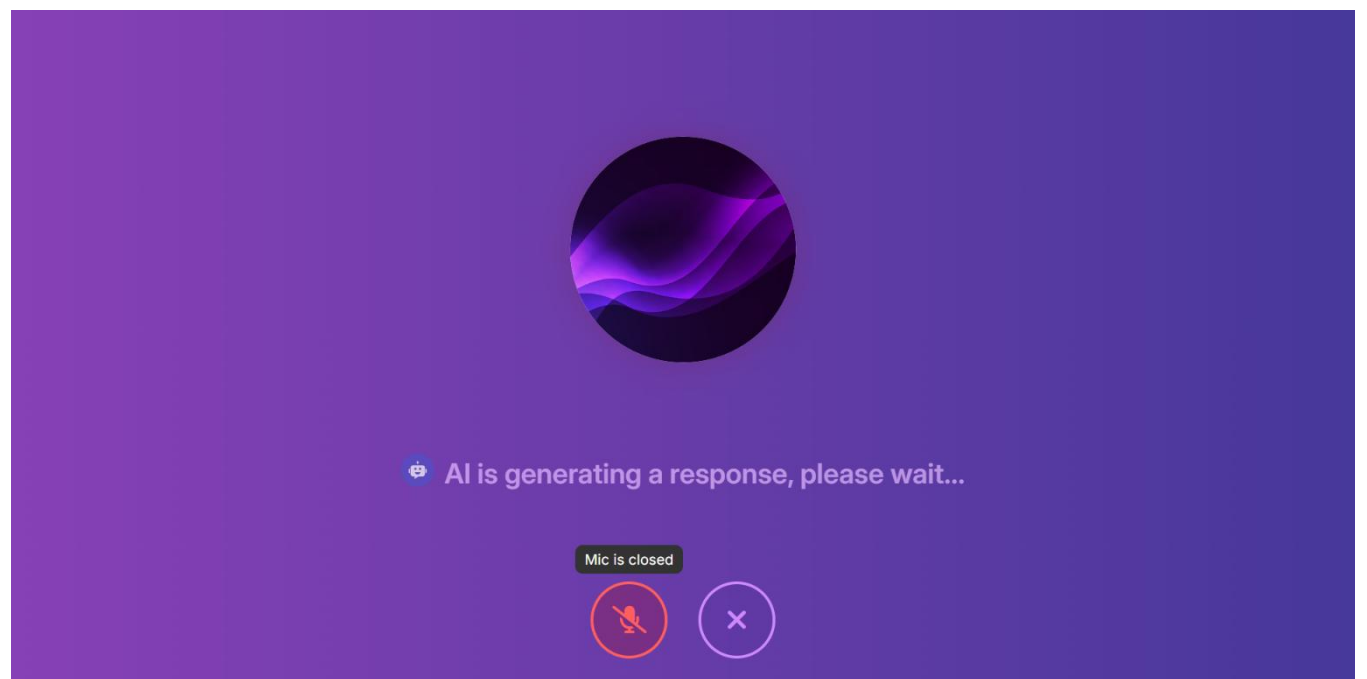
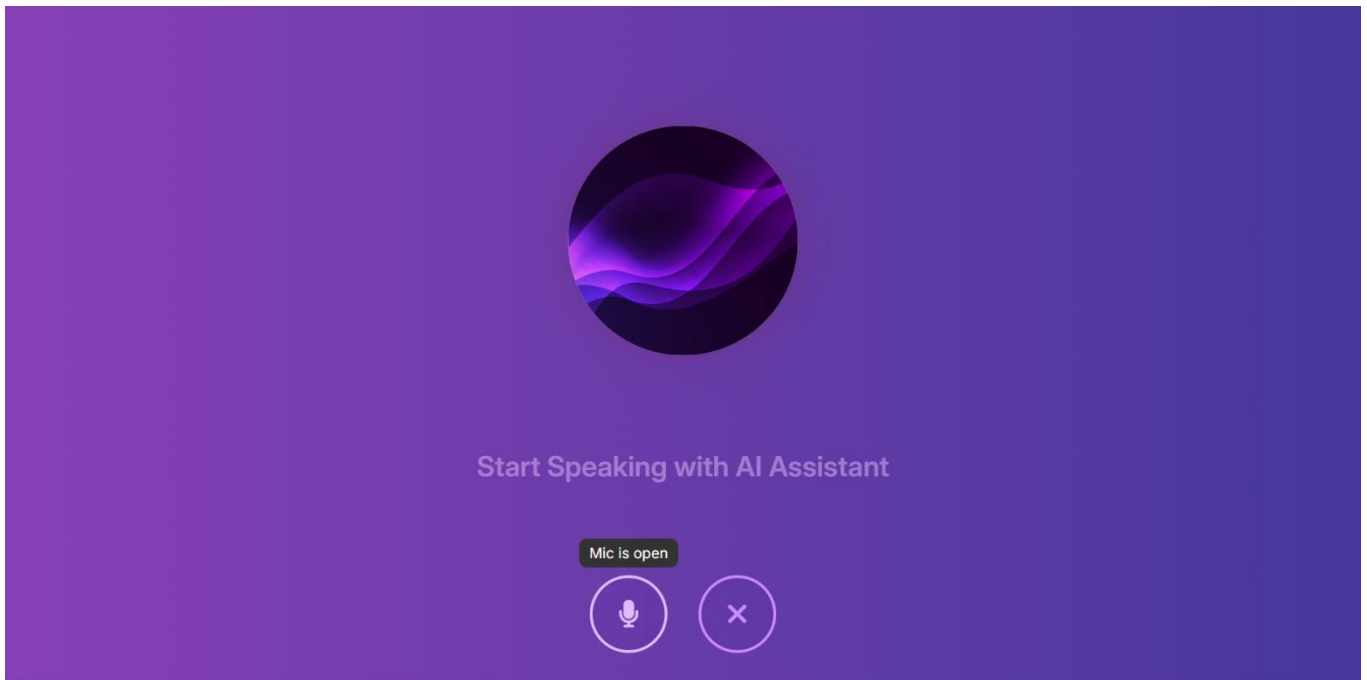




- Chat Mode



- **Voice Mode**



3.2 Backend Implementation

3.2.1 Database

The backend uses Entity Framework Core as the Object-Relational Mapper (ORM) to interact with a SQL Server database. Fluent API configurations are used extensively to define entity relationships, constraints, indexes, and cascading behaviors. The schema includes core tables like Users, Doctors, Patients, Admins, Chats, Chat Messages, and Refresh Tokens and AI Media Analysis data. Each user can have only one role, enforced structurally.

3.2.2 RESTful API Development (ASP.NET Core)

The system is built with ASP.NET Core Web API to expose RESTful endpoints for managing users, authentication, chat sessions, media uploads, and doctor verifications. Controllers follow a standard convention, and incoming request models are validated using FluentValidation to enforce data integrity. The API supports role-based access and ensures secure operations across endpoints.

Authentication Endpoints (/auth)

Method	Endpoint	Description
POST	/auth/register	Register with email and password
POST	/auth/verify-otp	Verify email using OTP
POST	/auth/resend-otp	Resend OTP to email
POST	/auth/login	Authenticate and receive tokens
POST	/auth/refresh-token	Generate a new access token
POST	/auth/logout	Revoke refresh token and logout
PUT	/auth/complete-profile	Submit profile info after verification

Chat Endpoints (/chat)

Method	Endpoint	Description	Access Role
POST	/chat/create	Create a new chat session with a title	Patient
PUT	/chat/update	Update the title of an existing chat	Patient
DELETE	/chat/delete	Soft delete a chat	Patient
GET	/chat/get-chats	Retrieve all chats for the authenticated patient	Patient
GET	/chat/get-chat-messages	Get all messages from a specific chat	Patient
GET	/chat/sync-messages	Sync new messages after a specific timestamp	Patient

3.2.3 SignalR-Based Real-Time Chat

Real-time communication between the patient and AI chatbot is implemented using SignalR. Patients can initiate chat sessions where messages are sent and received instantly. The SignalR hub acts as the bridge, broadcasting messages and managing connections.



SignalR Hub (/chatHub) methods

Method Name	Description
JoinChat	Allows a patient to join a chat based on the chat ID provided in the JoinChatDto request.
OnDisconnectedAsync	Handles user disconnection, removes the connection from the chat, and cleans up the connection-user map.
SendClientRequest	Sends a patient's request message, stores it, and broadcasts it to other devices in the chat.
ConfirmClientRequestAsync	Confirms the patient's message with message ID, client message ID, and timestamp.
ReceiveClientRequestAsync	Broadcasts the client's message to other devices in the same chat.
ReceiveAIResponseAsync	Sends the AI-generated response to all devices in the chat after generation

3.2.4 Emails & OTP Verification

During registration, email verification is handled through OTP (One-Time Password) codes. Secure alphanumeric codes are generated and stored in the database with expiration and consumption tracking. MailKit is used to send OTP emails to users. The MailKit package handles the SMTP connection, email composition, and sending, ensuring emails are delivered securely

3.2.5 JWT Authentication System

The authentication system is based on JWT (JSON Web Tokens) using the package **Microsoft.AspNetCore.Authentication.JwtBearer**, supporting both access and refresh tokens. Each refresh token is stored in the database and linked to a specific user. Tokens include expiration metadata, and refresh tokens can be revoked or rotated. The JWT payload includes role claims to enforce role-based authorization across the API.



3.2.6 AI Models Integrations

The backend integrates several AI models hosted on Modal to enhance the chatbot's functionality:

- Whisper: Speech-to-text conversion.
- Whisper SER: Emotion detection from voice.
- DeepFace: Facial emotion analysis from video/images.
- MentalLama Chat 7B: Generates AI responses from text input.
- Tortoise-TTS: Converts AI text responses to speech in voice mode.

Voice Interaction Mode

In voice mode, the patient speaks, and the AI responds in audio. Whisper transcribes voice to text, SER analyzes emotion, and Tortoise-TTS converts the AI's text response back to audio.

Normal Mode

For text messages or media files (future work for media file analysis), the AI responds in text. Media undergoes analysis (speech-to-text, emotion recognition, facial analysis), and results are fed into the LLM for a relevant response.

Orchestration Between Models

The LLM is stateless, meaning it doesn't retain context between requests. When a new message or media (e.g., voice, image, video) is sent by the patient, the relevant AI models process the input (such as speech-to-text or emotion analysis). The new input, along with the Chat ID, is passed to the LLM endpoint to retrieve the full context from the chat history file stored on disk for that specific chat.

The Modal endpoint loads the chat history file from disk using the Chat ID, appends the new input (whether it's text, emotion analysis, or speech-to-text) to the in-memory history, and sends the updated context to the LLM for response generation.

Once the AI response is generated, the backend stores the new message (including the analysis results and AI response) back in the corresponding chat history file on disk. This ensures that context is preserved for future interactions.

Why History Is Not Passed Through the ASP.NET Backend with Every User Input Instead of Storing It on Modal Hosting?

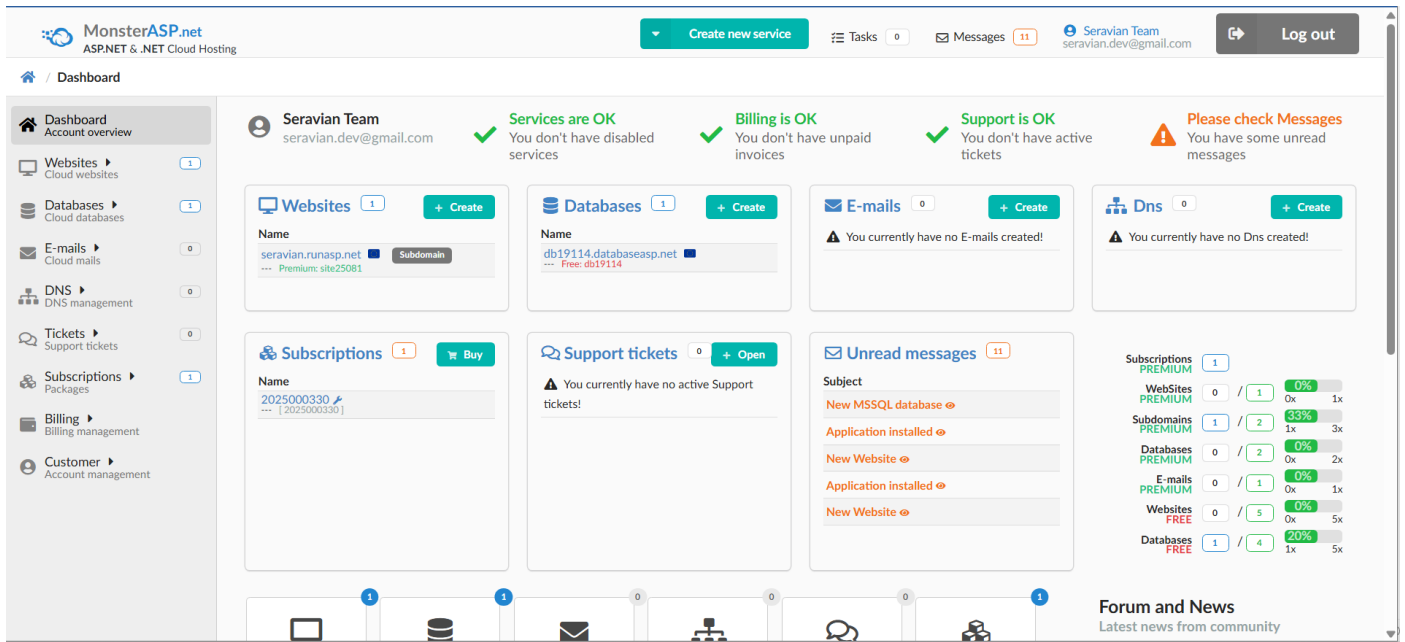
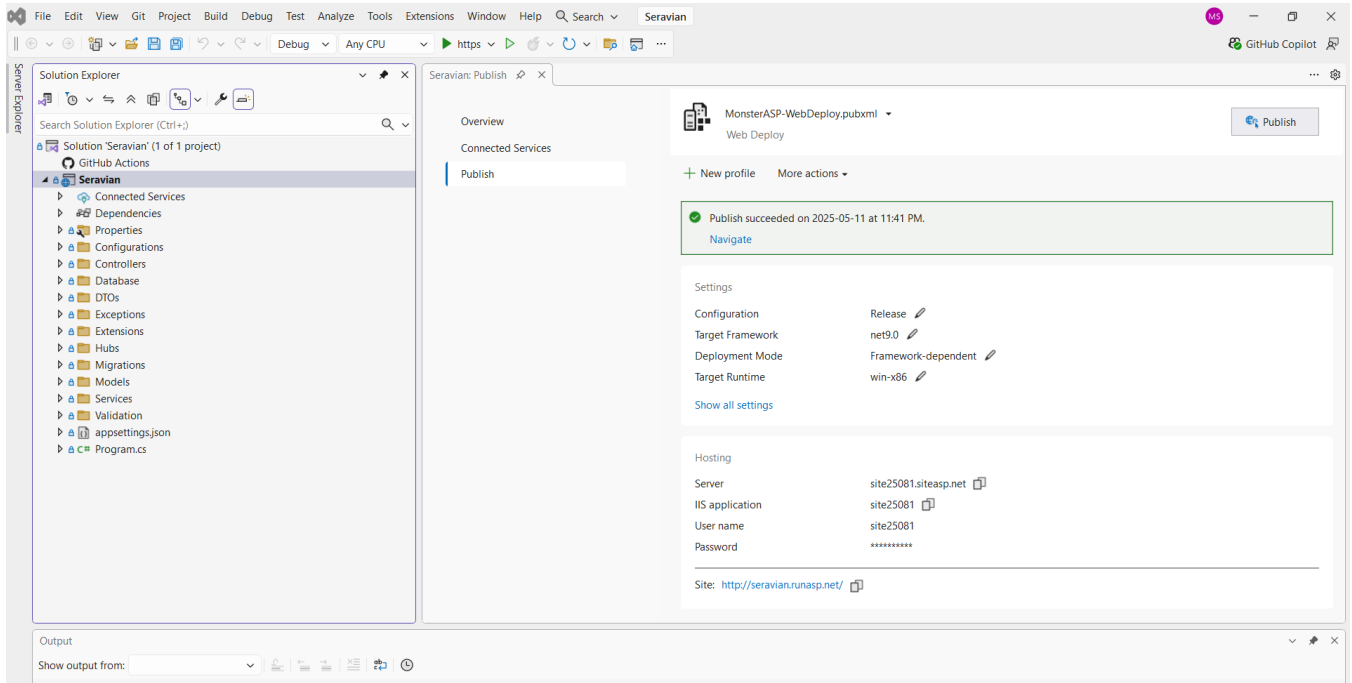
Performance & Bandwidth: Chat history could become very large over time. Constantly sending the entire history back and forth between the ASP.NET server and Modal would:

- Increase network latency.
- Consume unnecessary bandwidth.
- Add serialization/deserialization overhead.



3.2.7 Hosting and Deployment

The backend is hosted on MonsterASP Hosting, where environment variables are used for configuration in production (while user-secrets are used during development). Deployment is streamlined via Visual Studio publishing. The system uses HTTPS for secure communication, and API keys protect access to third-party AI services.



3.3. AI Models Integration

3.3.1 Chatbot with LLM

Training Phase – (kylang/mentallama-chat-7b)

Training Data: IMHI Dataset

Overview:

The IMHI (Instructional Mental Health Interactions) dataset is a curated collection of emotionally rich dialogues aimed at enhancing the emotional intelligence and empathy of conversational AI models.

Data Collection:

- Sourced from diverse social media platforms.
- Includes a broad range of human-to-human and human-to-AI dialogues.
- Special focus on:
 - Emotionally nuanced exchanges
 - Mental health support scripts
 - Empathetic prompts and responses
 - General question-answering conversations

Labeling & Structure:

- Dialogues are structured in instruction-response format, simulating multi-turn interactions.
- A subset of samples is annotated with high-level **intent tags**, such as:
 - seek_support
 - ask_info
 - vent_feelings
 - offer_advice
- These annotations help guide **instruction tuning** and **supervised fine-tuning (SFT)**.

Purpose:

Designed to improve the model's ability to detect user intent, respond empathetically, and handle mental health-related interactions with sensitivity and relevance.





```

1 from datasets import load_dataset
2
3 # Load dataset from CSV
4 dataset = load_dataset("csv", data_files="reddit_train.csv") # Replace with your file path
5
6 # Combine `post` and `question` as the input text
7 def preprocess_data(example):
8     example["input"] = f"Post: {example['post']} Question: {example['question']}"
9     return example
10
11 # Apply preprocessing
12 dataset = dataset.map(preprocess_data)

```

2. Preprocessing:

- **Tokenization:**
 - Convert input texts into token sequences using SentencePiece or Byte-Pair Encoding (BPE), consistent with the LLaMA tokenizer.
 - Example: "I'm feeling lost" → [223, 512, 4901]
- **Normalization:**
 - Convert to lowercase (handled by tokenizer).
 - Remove artifacts or special characters. ("?",", ".")
- **Text Cleaning:**
 - Remove HTML tags, corrupt characters, and apply sentence-level formatting.



```

1 input="I have been feeling down lately"
2 input_ids=tokenizer(input,return_tensors="pt").input_ids.to("cuda")

```



```
1 def tokenize_function(example):
2     encodings = tokenizer(example["post"], truncation=True, padding="max_length", max_length=512)
3     encodings["labels"] = encodings["input_ids"] # Set labels to input_ids for language modeling
4     return encodings
```

- **Formatting for Instruction Tuning:**

- Wrap conversations into instruction-completion format (e.g., Alpaca-style):
- **Instruction:**
 - I'm feeling overwhelmed.
- **Response:**
 - I'm here for you. Can you tell me more about what's been troubling you?

- **3. Feature Extraction:**

- **Embedding Preparation:**
 - Token embeddings are generated internally by the model's first layers.
- **Positional Encoding:**
 - Positional vectors help the transformer understand token order in context.
- **Semantic Learning:**
 - The model learns latent semantic features from conversation examples—no manual TF-IDF or BoW needed.

4. Training:

- **Model Architecture:**
 - Based on Meta's LLaMA 7B transformer model with ~7 billion parameters.



- **Fine-Tuning Process:**
 - Apply supervised fine-tuning on dialogue datasets with emotionally rich interactions.
 - Use DPO (Direct Preference Optimization) or SFT (Supervised Fine-Tuning).
- **Optimization:**
 - Optimizer: AdamW
 - Loss Function: Cross-Entropy (Token-level)
 - Training Scheduler: Cosine Decay / Linear Warmup
- **Batching:**
 - Use mixed precision and gradient accumulation for memory-efficient training.
- **Tools:**
 - Libraries: Hugging Face transformers, trainer, accelerate, bitsandbytes (for quantized training)

```

1 training_args = TrainingArguments(
2     output_dir="./llama-fine-tuned", # Output directory for model checkpoints
3     per_device_train_batch_size=8,
4     per_device_eval_batch_size=4,
5     gradient_accumulation_steps=16,
6     learning_rate=5e-5, # Set an appropriate initial learning rate
7     warmup_steps=500, # Linear warmup for the first 500 steps
8     weight_decay=0.01,
9     num_train_epochs=3,
10    fp16=True, # Use mixed precision for faster training
11    logging_dir="./logs",
12    logging_steps=10,
13    evaluation_strategy="steps",
14    eval_steps=100,
15    save_steps=200,
16    save_total_limit=3,
17    report_to="none", # Disable reporting (e.g., WandB or TensorBoard)
18 )

```





```
1  trainer = Trainer(  
2      model=model,  
3      args=training_args,  
4      train_dataset=tokenized_datasets["train"],  
5      eval_dataset=tokenized_datasets["validation"],  
6      tokenizer=tokenizer  
7  )
```

Prediction Phase – kyllang/mentallama-chat-7b

1. Input Message:

- **User Input:**
 - A raw textual message (optionally enriched with emotion tags or prior dialogue context).
 - Example: "I've been really anxious lately and can't focus."

2. Preprocessing:

- **Tokenization:**
 - The message is tokenized using the model's tokenizer.
- **Formatting:**
 - The prompt is wrapped in the expected instruction format, if necessary.

3. Feature Extraction:

- **Token Embeddings:**
 - Automatically handled by the model.
 - No external feature engineering required.



4. Inference:

- **Model Prediction:**
 - The input tokens are fed into the transformer model, which generates an autoregressive response.
 - Decoding uses strategies such as:
 - Greedy Sampling
 - Top-k Sampling
 - Top-p (Nucleus) Sampling
 - Temperature tuning (e.g., 0.7 for creativity control)
- **Response Generation:**
 - Output is returned as a generated string, representing the assistant's reply.

5. Result:

- **Output:**
 - The generated text is returned as the chatbot's response.
 - Example:
 - "I'm really sorry you're feeling this way. You're not alone, and I'm here to support you however I can. Would you like to talk more about what's on your mind?"

```
1 input="How to avoid a relapse? I've been having a particularly rough year; I attempted suicide"
2 input_ids=tokenizer(input,return_tensors="pt").input_ids.to("cuda")
3 outputs = mdl.generate(input_ids, max_length=256,num_return_sequences=1)
4 final=tokenizer.decode(outputs[0],skip_special_tokens=True)
5 print(final.split(final[-1]).strip())
```

Example Workflow:

- **1. User Input**
 - *"I feel like I'm not good enough."*
 - The user sends a message to the chatbot via the interface (e.g., web, mobile app, or voice-based input via Whisper).



- **2. Input Formatting**

- The message is embedded into a structured instruction format:
- **Instruction:**
- I feel like I'm not good enough.
- **Response:**

- **3. Tokenization**

- The input is tokenized into subword tokens using LLaMA's SentencePiece tokenizer.
- Example:
"I feel like I'm not good enough."
→ [29871, 1987, 2293, 29901, 293, 2248, 1311, 316]
- Each number represents a token ID in the model's vocabulary.

- **4. Embedding + Positional Encoding**

- Each token is converted into a dense vector (embedding).
- Positional encodings are added to help the model understand the sequence.

- **5. Attention Mechanism**

- The model uses **self-attention** to understand relationships between all tokens:
 - For every token, attention weights determine how much it should consider every other token in the sequence.
 - This allows the model to focus more on emotionally significant words like "not good enough".

- **6. Transformer Processing**

- The sequence of token embeddings is passed through 32 transformer layers (for LLaMA 7B).
- Attention mechanisms allow the model to consider all parts of the input context when generating each token in the response.

- **7. Autoregressive Generation**

- The model generates the response token-by-token, predicting the next most likely word given the previous context.
- Uses a decoding strategy such as **top-k sampling** or **temperature-controlled sampling** for balanced creativity and coherence.



- **8. Response Output**

- Response:
"I'm really sorry you're feeling this way. Remember, you're not alone and you deserve kindness and support—especially from yourself."
- This text is returned to the user through the chat interface.

- **9. Post-processing**

- Detokenization: Convert token IDs back to readable text.



```
1 response = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

3.3.2 Speech Recognition (Whisper)

Training Phase – openai/whisper-Large-V3 (Speech-to-Text)

1. Training Data:

- **Collection:**
 - Massive dataset of multilingual and multitask audio-transcription pairs.
 - Sources include web-crawled audio (e.g., TED talks, YouTube), podcasts, interviews, and more.
- **Labeling:**
 - Each audio clip is paired with:
 - A transcription (text)
 - Timestamp annotations (start and end per word/sentence)
 - Task type (e.g., transcribe, translate)
 - Language ID



2. Preprocessing:

- **Audio Processing:**
 - Normalize audio to 16 kHz sample rate and mono channel.
 - Convert raw waveform into **log-Mel spectrogram** input.
- **Text Normalization:**
 - Lowercasing
 - Removing punctuation and disfluencies
 - Standardizing numerals and abbreviations (e.g., “5 pm” → “five p m”)

3. Feature Extraction:

- **Spectrograms:**
 - Converts audio into 80-channel log-Mel spectrogram slices.
- **Tokenization:**
 - BPE (Byte Pair Encoding) tokenizer applied to output text for efficient multilingual modeling.
- **Multilingual Embedding Space:**
 - Whisper-v3 introduces better alignment across languages for improved zero-shot performance.

4. Model Architecture:

- **Encoder-Decoder Transformer:**
 - Large encoder processes audio spectrogram.
 - Decoder autoregressively generates tokenized output text.
- **Large V3 Enhancements:**
 - Optimized inference path and latency handling.
 - Improved token throughput (more real-time suitable).
- **Training Objective:**
 - Minimize cross-entropy loss between predicted and ground truth transcriptions or translations.



Prediction Phase – openai/whisper-large-v3

1. Input:

- Audio file or stream (WAV, MP3, FLAC) at 16 kHz.

2. Preprocessing:

- Convert audio to a normalized log-Mel spectrogram.

3. Inference:

- **Language Detection** (if not specified)
- **Task Selection:**
 - transcribe: Converts speech to text in the same language.
 - translate: Converts speech to English.
- **Autoregressive Decoding:**
 - Uses beam search or greedy decoding.
 - Decodes one token at a time using attention to entire audio context.

4. Output:

- Transcribed or translated text (with or without timestamps).

Example Workflow

1. **User Input (Audio):** A voice note — *"Hola, ¿cómo estás?"*
2. **Language Detection:** Spanish detected.
3. **Spectrogram Creation:** Audio converted into log-Mel spectrogram.
4. **Transformer Encoding:** Audio context embedded.
5. **Decoding:**
 - a. Task: translate
 - b. Output tokens: ["Hello", " ", "how", "are", "you", "?"]
6. **Response**

Output:

"Hello, how are you?"



3.3.3 Face Recognition with DeepFace

Overview

DeepFace is a deep learning facial recognition system developed by Facebook AI. It was one of the first systems to approach human-level accuracy on standard face verification benchmarks. DeepFace uses a deep convolutional neural network (CNN) trained on millions of face images to learn high-quality facial embeddings.

1. Training Phase

1.1 Dataset Collection

- **Data Source:** Millions of labeled face images from diverse identities.
- **Preprocessing:**
 - Face detection (e.g., using OpenCV or MTCNN).
 - Facial landmark detection and alignment to normalize pose.
 - Resize to a fixed dimension (e.g., 152×152 pixels).

1.2 Face Alignment

- DeepFace uses **3D face alignment** based on a 3D model to handle variations in pose.
- Aligns eyes, nose, and mouth to a canonical view, improving recognition accuracy.

1.3 Feature Extraction (CNN Architecture)

- A **9-layer deep neural network** with over 120 million parameters.
- Inspired by the AlexNet architecture but optimized for facial features.
- Input: Aligned face images.
- Output: A 4096-dimensional **embedding vector** representing the face in feature space.

2. Prediction (Inference) Phase

2.1 Face Detection & Alignment

- Detect the face in the input image.
- Align using 3D transformation.



2.2 Embedding Generation

- Pass the aligned face through the trained CNN.
- Output is a 4096-dimension feature vector.

2.3 Similarity Comparison

- Use **cosine similarity** or **Euclidean distance** between two embeddings:
 - If the distance is below a threshold, the faces are considered a match.

3. Example Workflow

1. **Input:** Two photos — one of the user and one from the database.
2. **Face Detection:** Localize and crop faces using a face detector.
3. **Distance Calculation:** Compute Euclidean or cosine distance.
4. **Verification:**
 - a. If distance < threshold → Match
 - b. Else → No match

4. Applications

- Face verification (1:1 match)
- Face recognition (1:N match)
- Emotion detection (with fine-tuned models)
- Identity authentication and security

3.3.4 Speech Emotion Recognition (SER) with Whisper

Overview

This system integrates **OpenAI's Whisper large-v3** model with a **custom emotion classification head**. It transcribes speech to text using Whisper's powerful encoder and simultaneously analyzes **emotional tone** from raw audio features using a secondary model trained on **emotion-labeled datasets**.



1. Architecture Overview

1.1 Whisper Large-v3 (Backbone)

- A **multilingual speech recognition transformer**.
- Produces:
 - **Text transcriptions**
 - **Audio features** (from internal transformer layers)

1.2 Emotion Classifier Head

- Takes embeddings from Whisper's encoder (e.g., final layer outputs).
- A feedforward or recurrent network maps audio features to emotion classes.
- Fine-tuned on datasets like **RAVDESS**, **CREMA-D**, or **IEMOCAP**.

2. Training Phase

2.1 Data Collection

- Use emotion-labeled datasets containing:
 - Raw audio
 - Emotion tags (e.g., happy, sad, angry, neutral, fear, surprise)

2.2 Preprocessing

- Convert raw audio to **log-Mel spectrograms**.
- Normalize audio to ensure consistency.
- Whisper automatically handles feature extraction using its pretrained encoder.

2.3 Model Setup

- Add a classification head:
 - Dense layers + dropout
 - Softmax activation for emotion classification
- Fine-tune on emotion-labeled speech data.

2.4 Loss Function

- **Categorical Cross-Entropy**: for multiclass emotion recognition.



3. Prediction Phase

3.1 Input Audio

- 16kHz mono waveform (standard format for Whisper)

3.2 Whisper Encoding

- Generates intermediate embeddings (log-Mel features → transformer outputs)

3.3 Emotion Classification

- Pass encoder output into emotion classifier head.
- Output: Probability distribution over emotion classes.

4. Example Workflow

1. **Input:** Audio file of someone saying “I’m really happy to see you!”
2. **Feature Extraction:** Whisper converts it into log-Mel spectrogram and runs it through its encoder.
3. **Embedding Capture:** Extract final-layer encoder outputs.
4. **Emotion Classifier:** Predicts emotion class → happy
5. **Transcription Output:** Optional output → “I’m really happy to see you!”

5. Supported Emotions

Depending on training data, typical emotion categories may include:

- Neutral
- Happy
- Sad
- Angry
- Fearful
- Disgusted
- Surprised



6. Applications

- **Call center analysis**
- **Mental health monitoring**
- **Smart assistants** that adapt based on user emotions
- **Human-computer interaction research**

3.3.5 Tortoise-TTS (Text-to-Speech)

Training Phase

Inputs:

- Large corpus of **text and paired audio** (speech recordings with transcripts)
- Multiple **voice samples** for speaker embeddings

Components Trained:

1. Autoregressive Transformer

- a. Learns to predict speech features (tokens or embeddings) from text + speaker identity

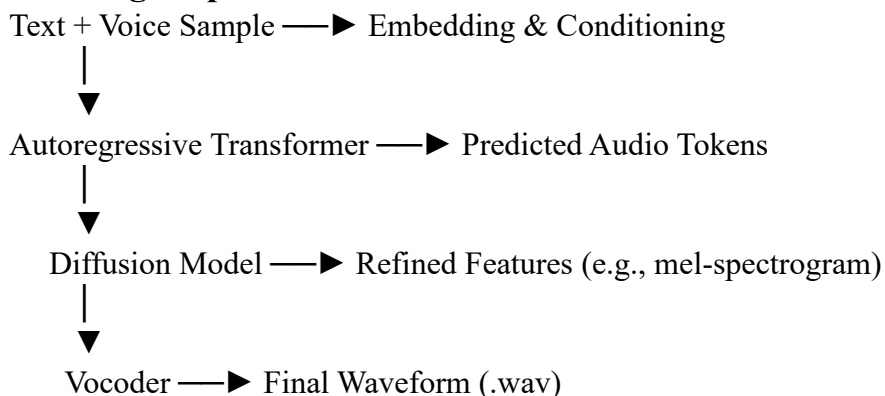
2. Diffusion Model

- a. Learns to refine noisy speech embeddings into high-quality versions

3. Vocoder (HiFi-GAN)

- a. Learns to synthesize waveforms from mel-spectrograms

Training Steps:



Prediction (Inference) Phase

Inputs:

- Text to synthesize
- Voice samples (can be real recordings or from existing voices)

Steps:

1. **Generate conditioning data** from voice samples (`load_voice`)
2. Pass text + voice into TextToSpeech:
 - a. Autoregressive model generates coarse speech representation
 - b. Diffusion model polishes it
 - c. Vocoder converts to waveform

Output:

- A **high-quality .wav file** representing natural-sounding synthesized speech.

TTS Example Workflow

Input

Text prompt: "I'm really happy to see you!"

Voice reference: 2–3 audio samples of a target speaker (e.g., `daniel.wav`, `daniel_2.wav`)

Voice Conditioning

Input voice samples are converted to latent vectors using Tortoise utilities.

- `voice_samples, conditioning_latents = load_voice("daniel")`
- `load_voice()` extracts speaker identity via internal speaker encoder.
- Latents are used to condition the TTS model on speaker style.

Text Encoding

Input text is tokenized and embedded.

- `text = "I'm really happy to see you!"`



- Tokens are passed into an **autoregressive transformer**.
- Text + voice conditioning are combined into a context-aware sequence.

Autoregressive Generation

The **Transformer decoder** generates a coarse sequence of audio features:

- Token-by-token generation conditioned on previous outputs.
- Produces a rough “skeleton” of how the sentence should sound.

Diffusion Refinement

The coarse audio features are **refined** using a **diffusion model**:

- Iteratively denoises the initial representation.
- Adds realism, smooth prosody, and naturalness.

Vocoder Synthesis

A **HiFi-GAN vocoder** converts the mel-spectrogram into raw waveform audio:

- `tts.tts_to_file(text, voice_samples, conditioning_latents, file_path="output.wav")`
- Final .wav file is saved with high-quality, expressive speech.

Output

Generated audio: A high-fidelity speech waveform of the text
spoken in the target speaker’s voice, expressing natural intonation.

output.wav: (*"I'm really happy to see you!"* (in the cloned voice of “daniel”))





```
1  from tortoise.api import TextToSpeech
2  from tortoise.utils.audio import load_voice
3
4  # Initialize model with optional speedups
5  tts = TextToSpeech(use_deepspeed=True, kv_cache=True)
6
7  # Load a preset voice (can also use your own audio clips)
8  voice_samples, conditioning_latents = load_voice("daniel")
9
10 # Synthesize and save audio
11 tts.tts_to_file(
12     text="This is an example of high-quality text to speech.",
13     voice_samples=voice_samples,
14     conditioning_latents=conditioning_latents,
15     file_path="output.wav"
16 )
17
```



CHAPTER FOUR:

Conclusion & Future Work



4.1 Conclusion

This project aimed to develop a comprehensive mental health platform that bridges users with licensed mental health professionals through both a website and an Android mobile application. The platform offers users multiple paths for mental health assistance, such as online doctor booking, a smart chatbot for AI-driven diagnosis, and a personalized experience based on their role—patient or doctor. The project prioritizes accessibility, security, and user-friendly design, providing users with the ability to sign up, verify their identity through OTP, and begin interacting with the platform immediately. Both the website and mobile app share a unified backend, ensuring consistent functionality and data synchronization across platforms. The mobile app offers an on-the-go experience while the web application focuses on broader accessibility and administrative features. The integration of frontend, backend, AI, and mobile components showcases a holistic approach to solving a real-world mental health access problem. Overall, the project lays a strong technical and functional foundation for scalable, impactful mental health care delivery.

4.2 Challenges Faced

During the development of the platform, we encountered a variety of challenges:

- **State Management in Stateless LLM:** Since LLMs are inherently stateless, we had to implement a custom system to handle conversation context. This involved:
 - Persisting chat history (including emotion analysis from media) externally and reloading it with each new request
 - Minimizing the performance and complexity overhead of transmitting full history with every new user message through the ASP.NET backend
- **Multimodal Input Processing Pipeline:** Orchestrating inputs from:
 - Voice (Whisper + SER)
 - Video (DeepFace)
 - Text (manual input)Coordinating emotion and transcription outputs before sending structured input to LLM
- **Cross-Platform Consistency:** Ensuring that both the web and mobile apps delivered a consistent experience in terms of user flow, data, and design required careful planning and constant testing.
- **Role-Based Access Control:** Designing logic and interfaces to support three distinct user types (patients, doctors and admins) across both platforms required a flexible and modular architecture.
- **Data Synchronization and Real-Time Updates:** Ensuring sessions, messages, and user data remained in sync across platforms demanded careful backend and API planning.
- **AI Chatbot Integration:** Making the AI chatbot work seamlessly on both the website and mobile app involved handling message formatting, input/output rendering, and performance optimization.



- **Real-Time Messaging:**

One of the main technical challenges was ensuring real-time communication across platforms. The system needed to support instant message delivery and synchronization between the web and mobile applications for users logged into the same account. For example, when a user sends a message from the mobile app, it should appear immediately on the web interface and vice versa. To achieve this, we implemented SignalR, a real-time communication library, which allowed us to maintain live connections and push updates across all active sessions without delay. Ensuring consistent performance and message integrity during these interactions was a critical technical milestone in the project.

Despite these difficulties, each challenge was met with practical solutions and contributed to a stronger final product.

4.3 Limitations of the System

Although the platform meets its core objectives, certain limitations remain in this version:

- **No Video Calling Feature:** Both mobile and web versions currently rely on chat. Real-time video sessions is a critical feature for modern mental health platforms are not yet implemented.
- **File Upload Missing in AI Chat:** Users cannot currently attach files (e.g., photos or documents) in conversations with the chatbot, which limits more complex interaction.
- **No Admin Panel in Mobile App:** Admin functionalities—such as monitoring users, managing reports, and verifying doctors—exist only on the website and not within the mobile app.
- **Model can NOT prescribe medication to users:** As model is only trained on data that identifies mental health problems, it can't prescribe medicine.
- **This project is not a replacement for a real psychiatrist or a doctor.**

These limitations highlight areas of focus for future development and refinement to better serve users and expand functionality.

4.4 Future Enhancements

Based on feedback, testing, and planned scalability, several enhancements are already identified for future development:

1. **Online Video Session Integration**

A major enhancement will be the addition of real-time video calling for booked sessions. This feature will be implemented using a cross-platform solution like WebRTC or Agora to ensure it works seamlessly on both web and mobile. It will significantly improve the platform's value by enabling face-to-face consultations.

2. **File Sharing in AI Chat**

Users will be able to upload and send files (images, videos and reports) during AI chat



interactions. This is essential for users needing to submit prior diagnoses or receive resources like mental exercises or prescriptions.

3. Admin Panel Improvements

The admin panel will be expanded to include mobile compatibility or at least a responsive web interface. This allows platform administrators to manage users, view reports, and handle complaints more flexibly.

4. Push Notifications & Reminders

Implementing push notifications for session reminders and doctor responses alerts will enhance user engagement across both platforms.

5. In-App Payments

A secure payment gateway will be added for booking paid consultations with doctors, allowing users to pay within the app or site using cards, wallets, or mobile money.

6. Video sessions with AI

Integrating the DeepFace model in the application to allow user to make video call sessions with the LLM model while also analyzing facial expressions and emotions seamlessly

By continuing to expand the features and usability of both the web and mobile platforms, the mental health system can grow into a more powerful tool for accessible, personalized care.



References

1. <https://www.sciencedirect.com/science/article/pii/S2949916X24000525>
2. https://journals.lww.com/indianjpsychiatry/fulltext/2023/65030/Artificial_intelligence_in_the_era_of_ChartGPT_.1.aspx
3. <https://www.ibm.com/docs/en/product-master/12.0.0?topic=processes-defining-use-cases>
4. Mentallama chat 7b model: <https://huggingface.co/klyang/MentaLLaMA-chat-7B>
5. Whisper large v3 model: <https://huggingface.co/openai/whisper-large-v3>
6. Speech emotion recognition model : <https://huggingface.co/firdhokk/speech-emotion-recognition-with-openai-whisper-large-v3>
7. DeepFace model: <https://github.com/serengil/deepface>
8. Tortoise text to speech model: <https://github.com/neonbjb/tortoise-tts>
9. Angular: <https://angular.dev/overview>
10. Bootstrap: <https://getbootstrap.com/docs/4.1/getting-started/introduction/>
11. Tailwind CSS: <https://tailwindcss.com/docs/>
12. Jetpack Compose: <https://developer.android.com/jetpack/compose>
13. Koin: <https://insert-koin.io/>
14. Ktor: <https://ktor.io/>
15. GetStream in Android: <https://getstream.io/video/docs/android/> (future work)
16. Retrofit: <https://square.github.io/retrofit/>
17. ASP.NET: <https://dotnet.microsoft.com/en-us/apps/aspnet>
18. SignalR: <https://dotnet.microsoft.com/en-us/apps/aspnet/signalr>
19. Microsoft SQL Server: <https://www.microsoft.com/en-us/sql-server>
20. Entity Framework (EF): <https://learn.microsoft.com/en-us/aspnet/entity-framework>
21. MonsterASP Hosting: <https://www.monsterasp.net/>
22. Modal AI Models Hosting: <https://modal.com/>

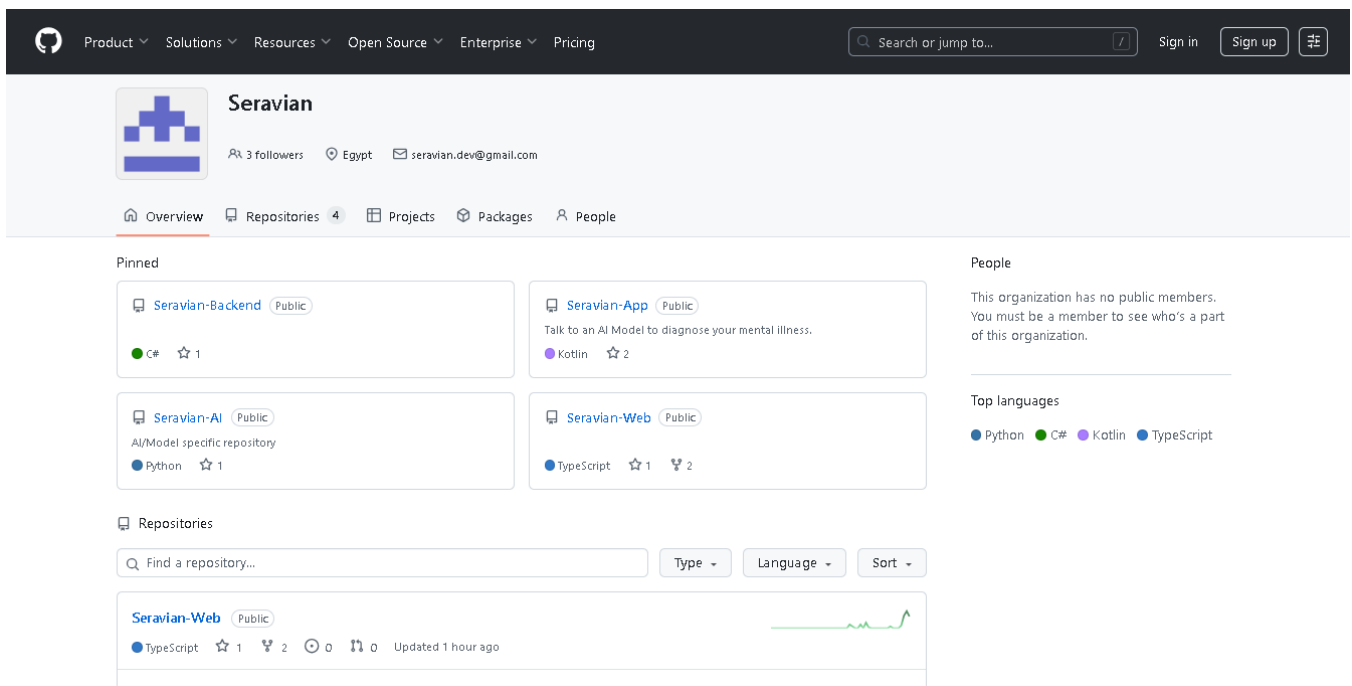


Appendix

The following images present the GitHub repositories that were developed and maintained throughout the project. Each repository contains the complete source code and relevant documentation for one of the core components of the system:

- [Seravian-AI](#)
- [Seravian-Backend](#)
- [Seravian-App](#)
- [Seravian-Web](#)

These repositories reflect the collaborative efforts of the team and demonstrate a structured approach to development. Their organization and content promote transparency, maintainability, and support for future enhancements.



Seravian / Seravian-App

Q

Type 7 to search

+

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

Settings

Seravian-App

Public

Edit Pins

Watch 0

Fork 0

Starred 2

production

6 Branches

0 Tags

Go to file

Add file

<> Code

GreenVenom77

Merge pull request #17 from Seravian/development

ad867fa · 10 hours ago

137 Commits

app	Enabled the Backhandler for the wanted screens	last week
auth	Enabled the Backhandler for the wanted screens	last week
chat	Added Edit and delete chat	10 hours ago
core-ui	Added Storing messages and chats locally in room and stori...	last week
crypto	Changed the need to token repository to Tokens datasource...	2 weeks ago
gradle	finished profile screen ui and logic	2 weeks ago
home	Enabled the Backhandler for the wanted screens	last week
local	Added Storing messages and chats locally in room and stori...	last week
navigation	Changed AR.Eg to AR changed some arabic translations and...	last week
network	Added the Mutex back to fix the bug where the client can't...	last week
onboarding	Enabled the Backhandler for the wanted screens	last week
profile	Changed AR.Eg to AR changed some arabic translations and...	last week
tokens	AI Chat now works as intended and my own messages show...	last week
validation	Removed Phone Number from the rquired data from the us...	3 weeks ago
.gitignore	Initialize project	5 months ago
LICENSE	Changed from License from Apache to Elastic V2	last month
build.gradle.kts	Moved storing tokens and refreshing them to datastore to it...	3 weeks ago
gradle.properties	Remodularized the project to almost industry level	2 months ago
gradlew	Initialize project	5 months ago
gradlew.bat	Initialize project	5 months ago
settings.gradle.kts	New Profile Repository with logic for theme and language s...	2 weeks ago

About

Talk to an AI Model to diagnose your mental illness.

kotlin-android

mobile-app

android-application

sessions

mental-health

mvi

mobile-development

psychiatry

koin

jetpack-compose

ktor-client

View license

Activity

Custom properties

2 stars

0 watching

0 forks

Report repository

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 2

GreenVenom77 Hossam Walid

Kessam10

Languages

Kotlin 100.0%

Seravian / Seravian-AI

Q

Type 7 to search

+

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

Seravian-AI

Public

Edit Pins

Watch 0

Fork 0

Starred 1

production

1 Branch

0 Tags

Go to file

Add file

<> Code

mohamedsaeed138

fix api key name in request header

79d1364 · 3 days ago

40 Commits

.gitignore	Add __pycache__ directory to gitignore	4 days ago
Copy of Mentallama.ipynb	Add files via upload	2 weeks ago
LICENSE	Create LICENSE	last month
deepface_modal.py	Host of deepface model on modal	last week
requirements.txt	Adds requirements.txt with project dependencies	last week
ser_modal_new.py	Add files via upload	4 days ago
seravian_llm_modal_fixed.py	Adds type hint for response string variable	4 days ago
tortoise_modal.py	fix api key name in request header	3 days ago

About

AI/Model specific repository

MIT license

Activity

Custom properties

1 star

0 watching

0 forks

Report repository

Releases

No releases published

Create a new release



Seravian-Web

Public

Edit Pins

Watch 0

Fork 2

Starred 1

chat

6 Branches

0 Tags

Add file

Code

alhrery2003 Merge pull request #9 from Seravian/Test 2354ee9 · yesterday 68 Commits		
public	chat-ui	2 weeks ago
src	all SignalR basic features are now fully working	yesterday
.editorconfig	Initialize-Project	5 months ago
.gitignore	.	2 weeks ago
.hintrc	chat-ui	2 weeks ago
.postcssrc.json	chat-ui	2 weeks ago
LICENSE	added license	3 weeks ago
README.md	Initialize-Project	5 months ago
angular.json	Update angular.json	2 weeks ago
firebase.json	Initialize-Project	5 months ago
package-lock.json	.	4 days ago
package.json	.	4 days ago
tailwind.config.js	chat-ui	2 weeks ago
tsconfig.app.json	Initialize-Project	5 months ago
tsconfig.json	Initialize-Project	5 months ago
tsconfig.spec.json	Initialize-Project	5 months ago

About

No description, website, or topics provided.

Readme

View license

Activity

Custom properties

1 star

0 watching

2 forks

Report repository

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 2

alhrery2003 Abdalrhman Alhrery

Khalidsaied

Deployments 6

Production

created 3 weeks ago



Seravian-Backend

Public

Edit Pins

Unwatch 1

Fork 0

Star 1

feature/chat

had recent pushes 45 minutes ago

Compare & pull request

feature/chat

3 Branches

0 Tags

Add file

Code

This branch is 19 commits ahead of, 2 commits behind main .

Contribute

mohamedsaeed138 add models: Chat Message Media, Chat Voice Analysis and their fl... ce82909 · 45 minutes ago 29 Commits		
Seravian	add models: Chat Message Media, Chat Voice Analysis and t...	45 minutes ago
.gitattributes	Initialize Seravian project with basic configuration and setup ...	2 months ago
.gitignore	update gitignore	2 weeks ago
LICENSE	Updates license from MIT to Elastic License 2.0	last month
README.md	Initialize Seravian project with basic configuration and setup ...	2 months ago
Seravian.slnx	Updates license from MIT to Elastic License 2.0	last month

About

No description, website, or topics provided.

Readme

View license

Activity

Custom properties

1 star

1 watching

0 forks

Report repository

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)


The following images showcase several RESTful API endpoints used in the project

The screenshot displays the Postman interface for a REST client. The top bar includes a search bar, a 'Ctrl K' shortcut, and buttons for 'Invite', 'Upgrade', and a minus sign. The main workspace shows a collection named 'Seravian | v1' with a sub-collection 'auth'. The selected endpoint is 'POST /auth/register'. The request body is a JSON object:

```
{  "email": "mohamed.saeed.goda.1313@gmail.com",  "password": "mohamed.saeed.goda.1313@M"}
```

. The response is a 200 OK status with a response time of 4.10 s and a body size of 321 B. The response body is a JSON object:

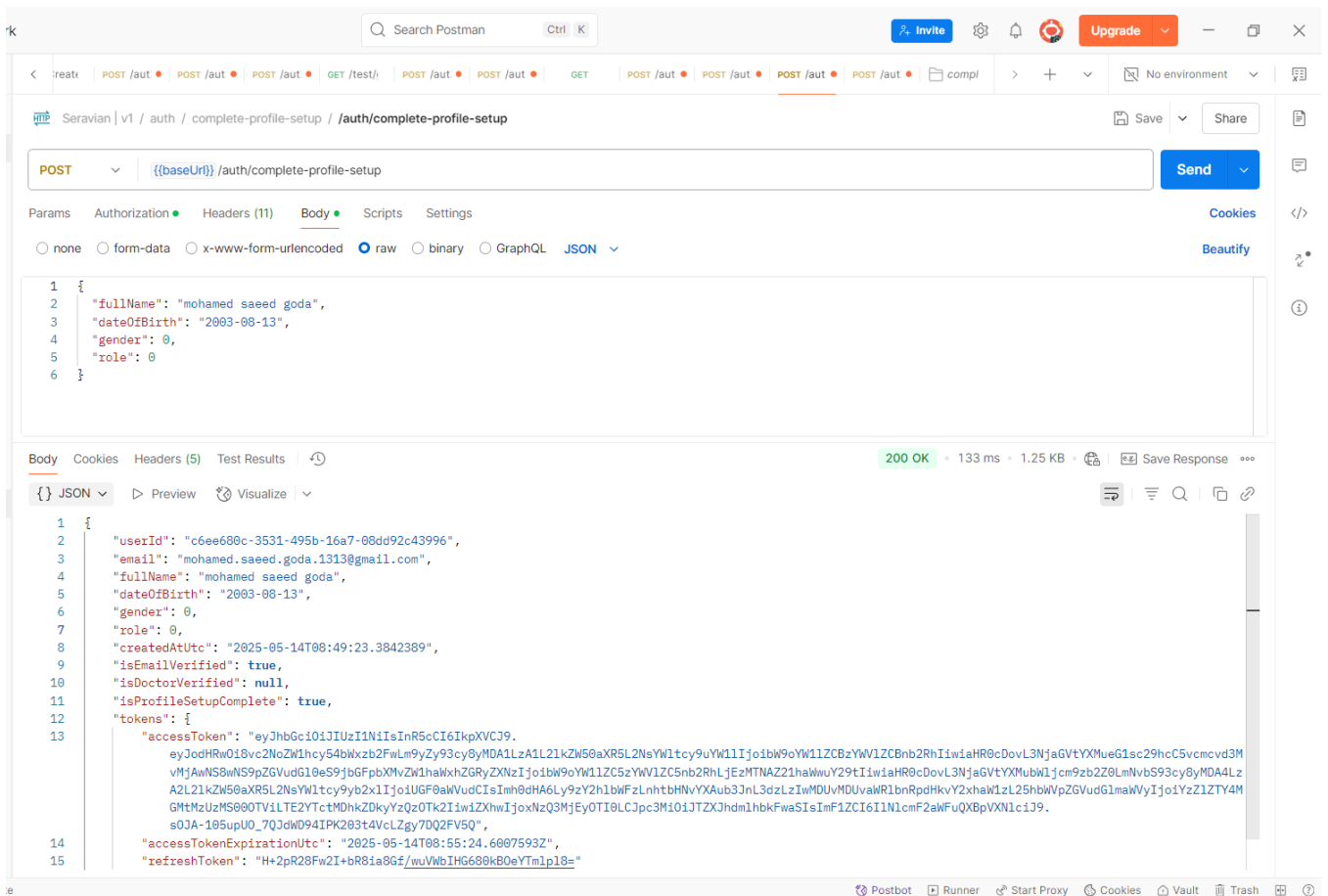
```
{  "userId": "c6ee680c-3531-495b-16a7-08dd92c43996",  "email": "mohamed.saeed.goda.1313@gmail.com",  "createdAtUtc": "2025-05-14T08:49:23.3842389Z"}
```

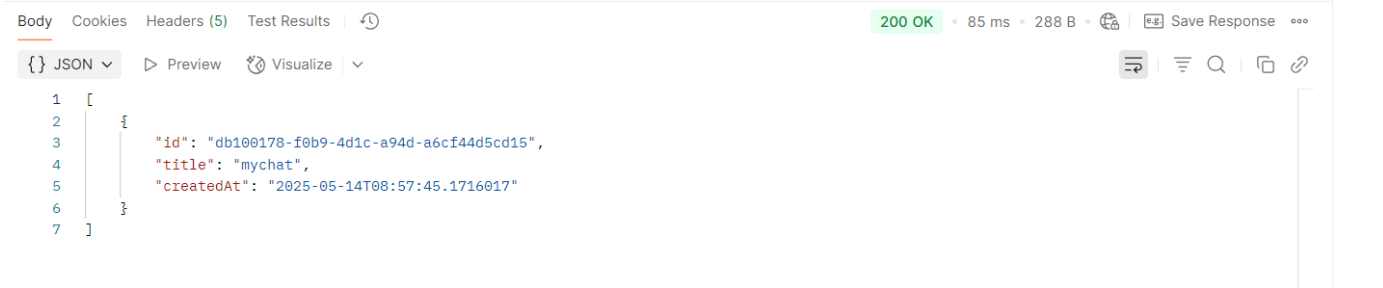
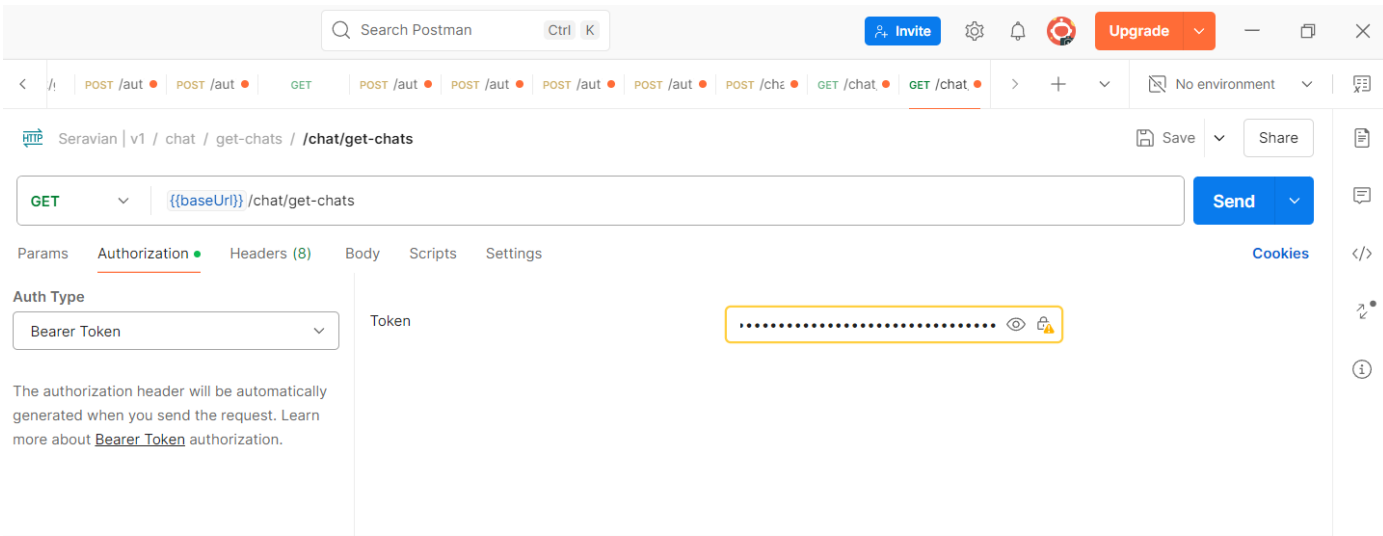
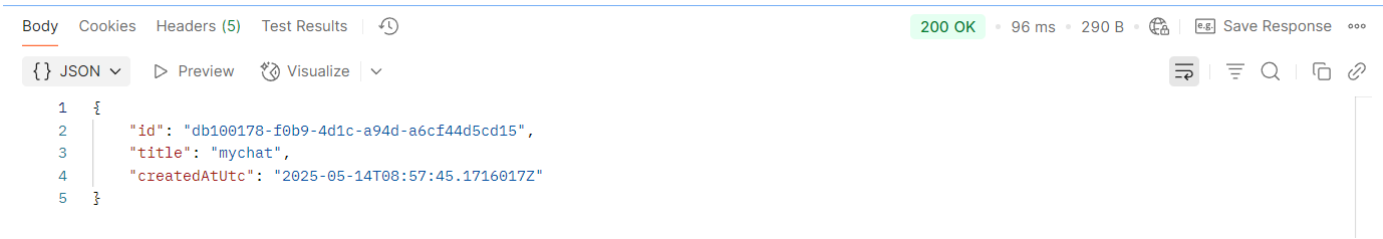
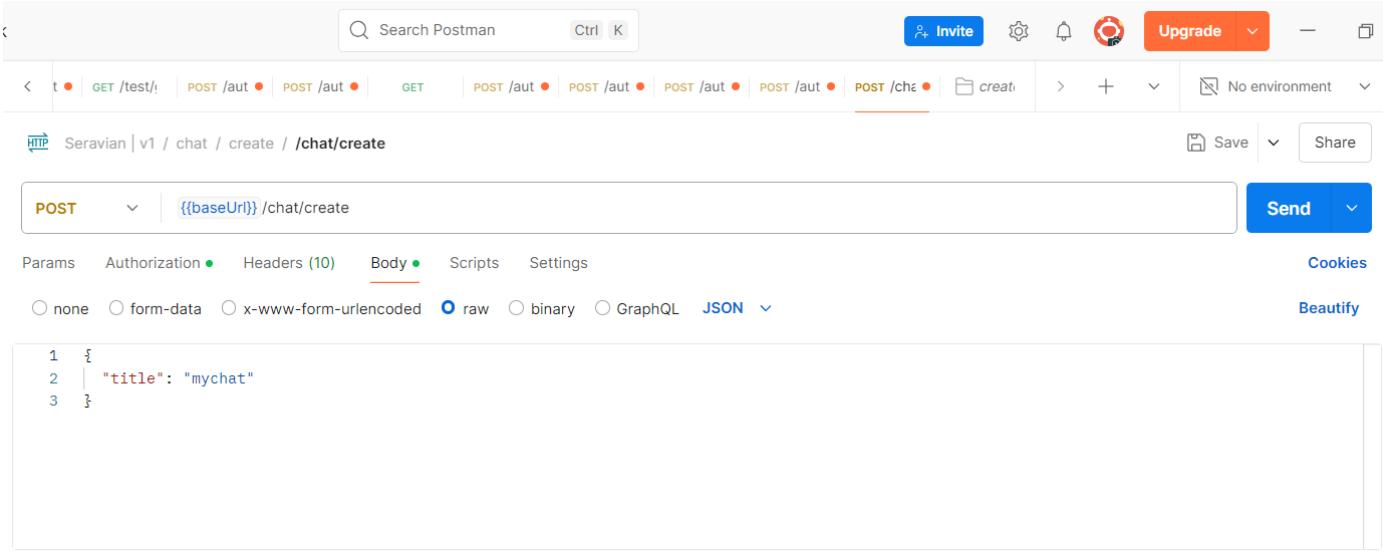
The screenshot displays the Postman interface for a REST client. The top bar includes a search bar, a 'Ctrl K' shortcut, and buttons for 'Invite', 'Upgrade', and a minus sign. The main workspace shows a collection named 'Seravian | v1' with a sub-collection 'auth'. The selected endpoint is 'POST /auth/verify-otp'. The request body is a JSON object:

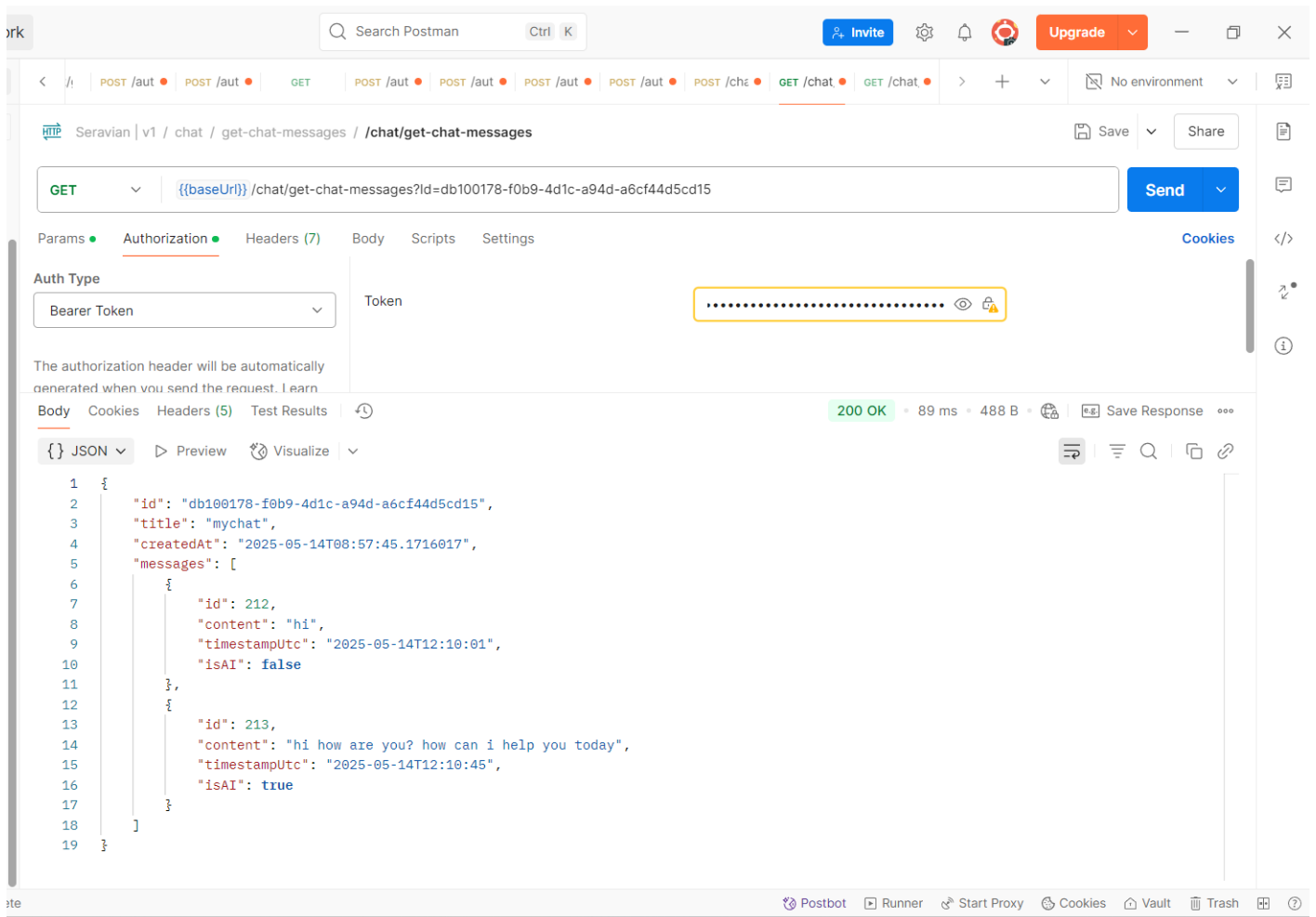
```
{  "email": "mohamed.saeed.goda.1313@gmail.com",  "otpCode": "H4FzEVqQ"}
```

. The response is a 204 No Content status with a response time of 502 ms and a body size of 115 B. The response body is empty.





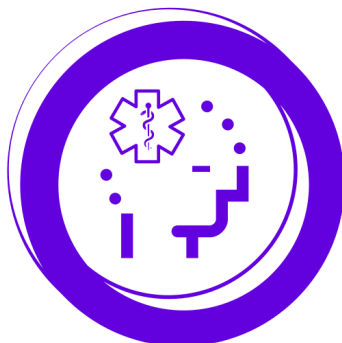




-WHY Seravian:-

Ser comes from the English word "serenity", which means peace or calmness.

Avian in Latin means bird, and this symbolizes freedom from depression, stress, and psychological disorders.



SERAVIAN



SERAVIAN

ملخص التقرير

يهدف هذا المشروع إلى تطوير منصة رقمية متكاملة تُعنى بتحسين خدمات الدعم النفسي باستخدام تقنيات الذكاء الاصطناعي المتقدمة. تتكون المنصة من تطبيق أندرويد وموقع إلكتروني يتيحان للمستخدمين التفاعل مع روبوت دردشة ذكي قادر على تقديم دعم نفسي أولي من خلال تحليل اللغة وتحديد الحالات النفسية بدقة.

يعتمد النظام على نموذج لغوي متقدم وهو MentalLLaMA 7B لفهم الأسئلة والاستجابات النفسية بدقة عالية، مع استخدام مكتبة HuggingFace من Transformers لتسهيل عمليات الترميز وفهم السياق. كما تم استخدام نموذج Whisper من OpenAI لتحويل الصوت إلى نص مما يتيح للمستخدمين إدخال استفسارات صوتية بشكل مباشر، ويُستخدم DeepFace لتحليل تعبيرات الوجه وتقدير الحالة العاطفية للمستخدم. تم الاستفادة أيضاً من مكتبة BitsAndBytes لضغط النموذج وتقليل استهلاك الموارد وتحسين الأداء على الأجهزة المحمولة.

يوفر النظام إمكانية تفاعل المستخدم مع روبوت الدردشة، كما يتيح للمستخدمين التسجيل كمرضى أو طبيب، حيث سٌضاف مستقبلاً إمكانية حجز المواعيد والتواصل المرئي بين الطرفين.

يعكس هذا المشروع خطوة مهمة نحو الاستفادة من الذكاء الاصطناعي في المجال النفسي، من خلال تقديم دعم فوري وسري وآمن للمستخدمين المحتاجين، وتقليل الضغط على الأخصائيين، وتعزيز الوصول إلى الرعاية النفسية من أي مكان وفي أي وقت.